

## **RAPPORT DE PROJET DE FIN DE MODULE**

POUR PROGRAMMATION ORIENTEE OBJET

### **MASTER SCIENCES ET TECHNIQUES**

**SPÉCIALITÉ : RÉSEAUX ET SYSTEME INFORMATIQUES**

---

# **Conception et Réalisation d'une Application de Gestion Hôtelière**

---

Réalisé par :

**Abdellatif Harakat  
ALMELLOUKI Nadir  
Saad Addar  
EL KHAIRI OMAR**

Sous la direction de :

**M. Said El Kafhali**, professeur à la FST de Settat

*Soutenu le :* **06 Janvier 2026**

**Année Universitaire : 2025-2026**

# Introduction Générale

L'industrie hôtelière évolue aujourd'hui dans un environnement hautement concurrentiel où la qualité de service et l'efficacité opérationnelle sont des facteurs clés de succès. La gestion manuelle ou fragmentée des réservations, des chambres et de la facturation ne suffit plus à répondre aux exigences de rapidité et de fiabilité attendues par les clients modernes. C'est dans ce contexte de transformation numérique que s'inscrit notre projet de fin de module.

Ce projet, réalisé dans le cadre du Master Sciences et Techniques, vise la conception et le développement d'une application de bureau complète pour la gestion hôtelière. L'objectif est double : répondre à une problématique métier concrète — la centralisation et l'automatisation des processus hôteliers — et mettre en pratique les concepts avancés de l'ingénierie logicielle, notamment la Programmation Orientée Objet (POO), les patrons de conception (Design Patterns) et l'architecture en couches.

L'application développée propose une solution intégrée permettant de gérer le cycle de vie complet d'une réservation, de l'arrivée du client à sa facturation, tout en assurant un suivi rigoureux de l'état des chambres et des performances de l'établissement. Elle s'appuie sur un socle technologique robuste alliant le langage Java, l'interface graphique JavaFX et le système de gestion de base de données MySQL.

Le présent rapport retrace les différentes étapes de réalisation de ce projet et s'articule autour de quatre chapitres principaux :

- Le **Chapitre 1 : Contexte général du projet** pose le cadre de l'étude, définit la problématique de la gestion hôtelière traditionnelle et présente les objectifs ainsi que la planification du projet.
- Le **Chapitre 2 : Étude du projet** détaille l'analyse des besoins fonctionnels et non fonctionnels, et justifie les choix technologiques et architecturaux opérés.
- Le **Chapitre 3 : Analyse et conception** présente la modélisation formelle du système à travers les diagrammes UML (Cas d'utilisation, Classes, Séquence) et la conception de la base de données.
- Le **Chapitre 4 : Réalisation** décrit la phase d'implémentation, l'environnement de développement, les interfaces réalisées ainsi que les tests effectués pour valider la solution.

**Chapitre 1 :**

**Contexte Général du Projet**

## **I. Introduction**

L'intégration des technologies de l'information au sein des processus de gestion est devenue un impératif pour garantir la compétitivité et l'efficacité des entreprises de services. Le secteur hôtelier, en particulier, se trouve confronté à des exigences croissantes en matière de rapidité, de fiabilité et de qualité de service. Dans ce cadre, la mise en place de systèmes d'information robustes constitue une réponse stratégique aux défis organisationnels modernes.

Le présent chapitre a pour vocation de définir le cadre général du projet de développement de l'application de gestion hôtelière. Il s'agit, dans un premier temps, de présenter la nature du projet, son contexte académique et son périmètre fonctionnel. Nous procéderons ensuite à une étude de l'existant, en analysant les limites des méthodes de gestion traditionnelles afin de dégager la problématique centrale. Enfin, nous détaillerons la planification du projet, en exposant les objectifs visés, le calendrier prévisionnel des tâches ainsi que l'organisation adoptée par l'équipe de développement. Cette phase introductive est essentielle pour comprendre les enjeux techniques et métiers qui ont guidé la conception et la réalisation de la solution logicielle présentée dans ce rapport.

## **II. Présentation générale du projet**

### **1. Description du projet**

Ce projet consiste en la conception et le développement d'une application de bureau dédiée à la gestion intégrale d'un établissement hôtelier. Il s'inscrit dans le cadre académique du module de Programmation Orientée Objet, dispensé au sein de la Faculté des Sciences et Techniques (FST) pour l'année universitaire 2025/2026. Sous la supervision du Professeur Said El Kafhali, ce travail vise à mettre en pratique les concepts fondamentaux du génie logiciel à travers la réalisation d'un système complet, fonctionnel et robuste.

#### **Nature et portée du système**

L'application développée a pour objectif principal la dématérialisation et l'automatisation des processus métiers de l'hôtel. Elle ne se limite pas à une simple gestion des réservations, mais propose une approche holistique couvrant l'ensemble des départements opérationnels. Le système permet ainsi de centraliser la gestion des chambres, le suivi des clients, la facturation, l'administration du personnel et la planification des tâches d'entretien. Cette centralisation offre une visibilité en temps réel sur l'activité de l'établissement, facilitant ainsi la coordination entre les différents services et la prise de décision.

#### **Socle technique et architecture**

D'un point de vue technologique, la solution repose sur le langage Java (version 25.0.1), choisi pour sa robustesse, sa portabilité et sa richesse en matière de bibliothèques. L'interface utilisateur est réalisée avec la technologie JavaFX (version 17), permettant de concevoir des écrans ergonomiques, modernes et réactifs. La persistance des données est assurée par le système de gestion de base de données relationnelle MySQL (version 8.0), garantissant l'intégrité et la sécurité des informations stockées.

L'architecture logicielle adoptée respecte rigoureusement le patron de conception MVC (Modèle-Vue-Contrôleur). Ce choix architectural assure une séparation claire entre la logique métier, l'interface graphique et l'accès aux données, favorisant ainsi la maintenabilité et l'évolutivité du code. De plus, le projet intègre plusieurs patrons de conception avancés (Design Patterns) tels que le DAO (Data Access Object) pour l'abstraction de la base de données, le Singleton pour la gestion unique des connexions, ainsi que les patterns Factory, Observer et Strategy pour répondre à des besoins spécifiques de conception modulaire.

## **Contexte pédagogique**

Ce projet représente l'aboutissement d'un apprentissage théorique et pratique. D'une durée intensive de quatre semaines, il simule les contraintes réelles d'un projet informatique professionnel. Il couvre l'intégralité du cycle de vie du logiciel, depuis l'analyse des besoins et la modélisation UML jusqu'à l'implémentation, les tests et la documentation technique. Il permet ainsi de valider l'acquisition des compétences en développement logiciel, en conception de bases de données et en gestion de projet.

### **2. Étude de l'existant**

L'analyse des méthodes de gestion traditionnelles dans le secteur de l'hôtellerie permet de mettre en lumière les inefficacités structurelles inhérentes aux systèmes manuels ou peu intégrés. Cette étude théorique se concentre sur les défis organisationnels couramment observés dans les établissements ne disposant pas d'un système d'information centralisé.

#### **Gestion manuelle et fragmentation de l'information**

Dans une approche traditionnelle, la gestion hôtelière repose souvent sur des supports papier (registres, plannings muraux) ou sur des outils bureautiques non interconnectés (tableurs isolés). Cette fragmentation des données entraîne une redondance de l'information, chaque département (réception, étage, comptabilité) maintenant ses propres registres. Il en résulte un risque élevé d'incohérence des données, où l'information disponible à la réception ne correspond pas nécessairement à la réalité du terrain, par exemple concernant l'état de propreté d'une chambre.

#### **Limites du suivi en temps réel**

L'absence de centralisation rend le suivi de la disponibilité des chambres particulièrement complexe. La mise à jour manuelle des plannings induit une latence entre la réservation effective et sa prise en compte globale. Cette situation favorise les erreurs critiques telles que le surbooking (double réservation pour la même chambre) ou, à l'inverse, le manque à gagner dû à une chambre apparaissant faussement occupée. De même, la coordination avec le service d'entretien est souvent ralentie par des échanges verbaux ou papier, retardant la remise en disponibilité des chambres pour les nouveaux clients.

#### **Risques liés à la facturation et à la sécurité**

Les processus de facturation manuels sont sources d'erreurs de calcul et d'oublis, notamment concernant les prestations annexes (services supplémentaires, consommations). L'absence d'automatisation prolonge les délais de check-out, impactant négativement l'expérience client. Par ailleurs, la sécurité des données sensibles (coordonnées clients, historiques) est difficilement garantie sur des supports physiques ou des fichiers locaux, exposant l'établissement à des risques de perte, de vol ou de détérioration de l'information.

### **3. Problématique**

Au regard de l'analyse de l'existant, il apparaît clairement que les méthodes de gestion traditionnelles ne permettent plus de répondre aux exigences d'efficacité et de fiabilité requises par le secteur hôtelier moderne. La problématique centrale de ce projet peut donc se formuler ainsi :

**Comment concevoir et réaliser un système d'information intégré et sécurisé permettant d'optimiser l'ensemble des processus opérationnels d'un hôtel, de la réservation à la facturation, tout en garantissant l'intégrité des données et la fluidité de l'expérience utilisateur ?**

Cette problématique générale se décline en plusieurs enjeux spécifiques auxquels notre solution doit répondre :

- **L'efficacité opérationnelle** : Comment réduire les tâches manuelles répétitives et chronophages pour permettre au personnel de se concentrer sur le service client ?
- **L'intégrité des données** : Comment assurer la cohérence de l'information entre les différents services et éviter les conflits de données (doublons, incohérences de statut) ?
- **La visibilité en temps réel** : Comment offrir une vue instantanée et précise sur l'état de l'hôtel (taux d'occupation, chambres en maintenance) pour faciliter le pilotage de l'établissement ?
- **La sécurité et la traçabilité** : Comment protéger l'accès au système et assurer un suivi rigoureux des actions effectuées par les différents utilisateurs ?
- **L'évolutivité** : Comment structurer l'application pour qu'elle puisse s'adapter aux évolutions futures des besoins de l'établissement ?

C'est pour répondre à ces questions que le projet de conception et de développement de l'application de gestion hôtelière a été initié.

### III. Planification du projet

La réussite d'un projet de développement logiciel repose sur une définition claire des objectifs et une planification rigoureuse des tâches. Cette section détaille les buts visés ainsi que l'organisation temporelle et humaine mise en œuvre.

#### 1. Objectifs à atteindre

Le projet s'articule autour de quatre catégories d'objectifs complémentaires.

#### Objectifs fonctionnels

L'application doit couvrir l'ensemble du périmètre métier défini dans le cahier des charges :

- Développer une application de bureau modulaire assurant la gestion complète du cycle de vie des réservations (création, modification, annulation).
- Mettre en œuvre une gestion dynamique du parc de chambres, incluant les statuts de disponibilité, d'occupation et de maintenance.
- Assurer la gestion administrative des clients avec un historique détaillé de leurs séjours.
- Intégrer un module de facturation automatisé capable de générer des documents au format PDF.
- Fournir un tableau de bord décisionnel présentant les indicateurs clés de performance (KPI) en temps réel.
- Permettre la gestion des droits d'accès via des rôles utilisateurs distincts (administrateur, réceptionniste).

#### Objectifs techniques

La réalisation technique doit respecter les standards de qualité académiques et professionnels :

- Appliquer strictement les paradigmes de la programmation orientée objet (encapsulation, héritage, polymorphisme).
- Structurer l'application selon l'architecture en couches MVC pour découpler l'interface, la logique métier et les données.

- Implémenter de manière pertinente les Design Patterns étudiés (Singleton, Factory, DAO, Observer, Strategy).
- Assurer la persistance fiable des données via JDBC et MySQL, en respectant les contraintes d'intégrité référentielle.
- Concevoir une interface utilisateur riche et intuitive en utilisant la bibliothèque JavaFX et le langage FXML.

### Objectifs de qualité

La robustesse et la fiabilité de l'application sont des priorités :

- Garantir la sécurité des authentifications par le hachage des mots de passe (algorithme SHA-256) et des mécanismes de protection contre les attaques par force brute.
- Optimiser les performances d'accès aux données grâce à l'utilisation d'un pool de connexions (bibliothèque HikariCP).
- Assurer une expérience utilisateur fluide grâce à la validation des saisies en temps réel et des retours visuels explicites.
- Produire un code lisible, documenté et respectant les conventions de nommage standard.

### Objectifs pédagogiques

Au-delà du produit final, le projet vise le développement de compétences :

- Maîtriser la modélisation UML pour l'analyse et la conception de systèmes complexes.
- Expérimenter le travail collaboratif et l'utilisation d'outils de gestion de versions (Git/GitHub).
- Appréhender la gestion du temps et des priorités dans un contexte de délais contraints.
- Renforcer les capacités de rédaction technique et de synthèse.

## 2. Planning prévisionnel

La gestion du projet a suivi une méthodologie itérative inspirée des méthodes agiles, structurant le développement en quatre sprints d'une durée d'une semaine chacun. Ce découpage a permis de valider progressivement les briques techniques et fonctionnelles de l'application.

- **Sprint 0 : Analyse des besoins (Semaine 1)** Cette phase de cadrage a été consacrée à l'étude théorique du système. L'objectif principal était de formaliser le périmètre du projet à travers l'identification exhaustive des besoins fonctionnels et non-fonctionnels. C'est durant cette semaine qu'ont été conçus le Modèle Conceptuel de Données (MCD) et les diagrammes UML structurants (cas d'utilisation, classes), posant les bases de l'architecture.
- **Sprint 1 : Fondations (Semaine 2)** Ce sprint a marqué le début du développement technique avec la mise en place du socle de l'application. Les travaux se sont concentrés sur la création de la base de données relationnelle et la configuration de la couche de persistance (connexion JDBC). L'équipe a implémenté les entités du modèle (Java Beans) ainsi que les premières classes d'accès aux données (DAO), notamment le module de gestion des chambres.
- **Sprint 2 : Logique Métier (Semaine 3)** La troisième semaine a été dédiée à l'implémentation du cœur fonctionnel de l'application. Ce sprint a permis de développer les services métier complexes (Service Layer), incluant la logique de gestion des réservations, le moteur de facturation et les algorithmes de vérification de disponibilité. Les mécanismes de validation des données ont également été intégrés à cette étape pour garantir l'intégrité du système.

- **Sprint 3 : Interface & Tests (Semaine 4)** La phase finale a porté sur la couche de présentation et la stabilisation du produit. Elle a englobé le développement des interfaces graphiques JavaFX et l'intégration du tableau de bord décisionnel (Dashboard). En parallèle, une campagne de tests (unitaires et d'intégration) a été menée pour valider le bon fonctionnement de l'ensemble, clôturée par la rédaction de la documentation technique finale.

| Sprint                                  | Durée     | Objectifs principaux                                                           |
|-----------------------------------------|-----------|--------------------------------------------------------------------------------|
| <b>Sprint 0 : Analyse des besoins</b>   | Semaine 1 | Identifier besoins fonctionnels et non-fonctionnels, MCD, diagrammes UML       |
| <b>Sprint 1 : Fondations</b>            | Semaine 2 | Création BDD, connexion JDBC, entités modèles, DAO chambre                     |
| <b>Sprint 2 : Logique Métier</b>        | Semaine 3 | Services métier (chambres, réservations, facturation), validation, algorithmes |
| <b>Sprint 3 : Interface &amp; Tests</b> | Semaine 4 | Vues JavaFX, Dashboard, tests unitaires, intégration, documentation            |

### 3. Répartition des tâches

Pour mener à bien ce projet dans les délais impartis, l'équipe a adopté une méthodologie de travail collaborative, fondée sur une répartition équilibrée des responsabilités et une communication constante.

Les tâches ont été distribuées non seulement par module fonctionnel mais aussi selon les phases techniques du projet. Cette approche a permis une progression parallèle efficace : pendant qu'une partie des ressources se consacrait à la conception et à la base de données, l'autre pouvait initier le maquetage des interfaces et la structure du code.

La méthodologie a favorisé des échanges réguliers pour assurer la cohérence architecturale de l'ensemble. Les responsabilités ont été attribuées en tenant compte des affinités techniques de chaque membre, tout en veillant à ce que chacun garde une vision globale du système. Certains membres ont ainsi pris le leadership sur l'analyse UML et l'architecture de la base de données (correspondant aux chapitres d'analyse et de conception), tandis que d'autres se sont focalisés sur l'implémentation de la logique métier complexe, l'interface graphique et l'intégration technique (correspondant aux chapitres de réalisation).

L'utilisation d'outils de collaboration modernes a été centrale dans notre organisation. Le système de gestion de versions Git, couplé à la plateforme GitHub, a permis de centraliser le code source, de gérer les conflits de modification et d'assurer un historique précis de l'avancement. Des points d'étape fréquents ont été organisés pour synchroniser les travaux, arbitrer les choix techniques et ajuster le planning en fonction des difficultés rencontrées.

## IV. Conclusion

Ce premier chapitre a permis de poser les fondations contextuelles indispensables à la bonne compréhension du projet. Nous avons défini le périmètre de l'application de gestion hôtelière, identifié les contraintes académiques et techniques, et mis en lumière les limites des systèmes traditionnels qui justifient notre démarche de développement. La problématique a été clairement énoncée autour de la nécessité d'un système intégré, fiable et performant.



La planification présentée et la méthodologie collaborative adoptée constituent le cadre structurant qui a guidé l'ensemble de la réalisation. Ces éléments préliminaires étant établis, il convient désormais d'entrer dans le cœur technique du sujet. Le chapitre suivant sera ainsi consacré à l'analyse approfondie et à la conception du système, détaillant la modélisation UML et les choix architecturaux qui sous-tendent notre solution.

# Chapitre 2 :

## Étude du projet

## **I. I. Introduction**

Suite à la présentation du contexte général et de la problématique dans le chapitre précédent, il convient à présent d'approfondir l'analyse des exigences du système afin de définir avec précision le périmètre de la solution envisagée. Cette étape, charnière entre l'expression initiale des besoins et la conception détaillée, vise à transformer les objectifs métiers en spécifications techniques tangibles et mesurables.

Ce chapitre, intitulé "Étude du projet", adopte une démarche structurée en deux temps. Dans une première partie, nous mènerons une étude fonctionnelle rigoureuse, identifiant l'ensemble des services que le système doit rendre à ses utilisateurs ainsi que les interactions prévues. Cette analyse permettra de circonscrire les fonctionnalités essentielles de gestion hôtelière. Dans une seconde partie, nous procéderons à une étude technique approfondie, définissant les contraintes non fonctionnelles telles que la sécurité et la performance, et justifiant les choix architecturaux et technologiques opérés. L'objectif est de démontrer la cohérence de nos décisions techniques vis-à-vis des enjeux académiques et professionnels identifiés.

## **II. II. Étude fonctionnelle**

L'étude fonctionnelle constitue le fondement de tout développement logiciel réussi. Elle a pour vocation de traduire les besoins exprimés par les parties prenantes en un ensemble cohérent de fonctionnalités logicielles, garantissant ainsi que le produit final réponde aux attentes opérationnelles des futurs utilisateurs.

### **1. 1. Besoins fonctionnels**

L'analyse détaillée du cahier des charges a permis de décomposer le système en plusieurs modules fonctionnels distincts mais interconnectés, couvrant l'intégralité du cycle d'exploitation hôtelier.

#### **1.1. Gestion de l'authentification et des accès**

La sécurité étant une préoccupation majeure, le module d'authentification constitue la porte d'entrée obligatoire du système. L'application doit garantir que seuls les utilisateurs dûment identifiés peuvent accéder aux fonctionnalités. Le système doit ainsi permettre une connexion sécurisée via un couple identifiant et mot de passe. Au-delà de la simple authentification, une gestion fine des droits d'accès est indispensable. L'application doit distinguer les rôles, notamment celui d'administrateur, disposant de privilèges étendus, et celui de réceptionniste, focalisé sur les opérations courantes. Pour prévenir les accès non autorisés, un mécanisme de sécurité active doit être implémenté, tel que le blocage temporaire du compte après plusieurs tentatives de connexion infructueuses.

#### **1.2. Gestion des chambres**

Le cœur de métier de l'hôtel reposant sur la location d'espaces, le module de gestion des chambres doit offrir une vision exhaustive et dynamique du parc immobilier. Les utilisateurs doivent être en mesure de consulter la liste des chambres disponibles, filtrables selon des critères pertinents comme la catégorie (simple, double, suite) ou le statut actuel. L'administrateur doit disposer des outils nécessaires pour maintenir cet inventaire à jour, incluant la création, la modification ou la suppression de fiches chambres. Un aspect critique de ce module réside dans la gestion du cycle de vie des chambres : le système doit gérer dynamiquement les transitions d'états, passant de "disponible" à "occupée", "réservée" ou "hors service". La gestion de la maintenance est particulièrement cruciale, nécessitant une fonctionnalité dédiée pour bloquer les réservations sur une chambre nécessitant des réparations, assurant ainsi la fiabilité du planning.

### **1.3. Gestion des réservations**

Ce module représente le centre nerveux de l'activité commerciale. Le système doit permettre la création intuitive de nouvelles réservations, associant un client à une ou plusieurs chambres pour une période donnée. L'intelligence du système réside ici dans sa capacité à vérifier automatiquement la disponibilité des ressources, détectant et empêchant tout conflit de dates ou double réservation. Le calcul du coût du séjour doit être automatisé, prenant en compte le tarif des nuitées ainsi que les éventuels services additionnels. Le cycle de vie d'une réservation doit être géré intégralement, incluant la possibilité de modification des dates, d'annulation, ainsi que les opérations de check-in à l'arrivée et de check-out au départ. Cette flexibilité est indispensable pour gérer les aléas quotidiens tels que les prolongations de séjour ou les départs anticipés.

### **1.4. Gestion de la clientèle**

Afin d'optimiser la relation client et de fluidifier les processus d'accueil, le système doit centraliser les informations relatives aux hôtes. Il est nécessaire de pouvoir enregistrer les coordonnées complètes des nouveaux clients et de consulter l'historique de leurs séjours précédents. Une fonctionnalité de recherche performante doit permettre au réceptionniste de retrouver rapidement une fiche client existante pour y associer une nouvelle réservation, évitant ainsi la saisie redondante d'informations et les doublons dans la base de données.

### **1.5. Gestion de la facturation**

Le volet financier exige une rigueur absolue. Le module de facturation doit automatiser la génération des documents comptables à l'issue du séjour. Le système doit être capable d'agréger l'ensemble des coûts, incluant les nuitées et les consommations de services divers, pour produire une facture détaillée et juste. Une exigence fonctionnelle majeure est la capacité d'exporter ces documents sous un format standardisé et inaltérable, tel que le PDF, pour remise au client ou archivage numérique. Le système doit également assurer le suivi des statuts de paiement, permettant de distinguer clairement les factures réglées de celles en attente de solde.

### **1.6. Tableau de bord et statistiques**

Pour supporter la prise de décision managériale, l'application doit offrir une vue synthétique de l'activité. Un tableau de bord doit présenter en temps réel les indicateurs clés de performance (KPI), tels que le taux d'occupation actuel ou le chiffre d'affaires généré sur la période. Ces outils de pilotage permettent à la direction d'avoir une vision claire de la santé financière et opérationnelle de l'établissement sans avoir à consolider manuellement des données éparses.

### **1.7. Gestion du personnel et de la maintenance**

Enfin, le système doit intégrer des fonctionnalités de gestion interne. L'administration des comptes employés permet de gérer les équipes et leurs droits d'accès. Parallèlement, le suivi des interventions techniques est assuré par un module de maintenance, permettant de planifier les tâches d'entretien et de garder une trace historique des réparations effectuées sur chaque chambre, contribuant ainsi à la pérennité des infrastructures.

## **2. Cas d'utilisation**

Pour mieux appréhender la dynamique du système, il est essentiel de définir les acteurs qui interagissent avec lui et de décrire les scénarios d'utilisation les plus significatifs.

## 2.1. Spécification des acteurs

L'analyse du système met en évidence deux acteurs principaux, chacun possédant un périmètre de responsabilité distinct.

**Le Personnel (Réceptionniste)** Le Personnel, représenté principalement par les réceptionnistes, constitue l'acteur opérationnel central du système. Ces utilisateurs sont en première ligne de la gestion quotidienne de l'hôtel. Leurs interactions avec l'application sont fréquentes et concernent l'ensemble du cycle client : de la prise de réservation téléphonique à l'accueil physique, en passant par la gestion des demandes en cours de séjour et l'encaissement final. Leurs droits d'accès sont configurés pour leur permettre d'effectuer toutes les tâches commerciales et administratives nécessaires au bon fonctionnement de la réception, tout en restreignant l'accès aux fonctions de configuration critique du système.

**L'Administrateur** L'Administrateur occupe un rôle de supervision technique et managériale. Il dispose de privilèges étendus lui conférant un contrôle total sur l'application. Au-delà des fonctionnalités accessibles aux réceptionnistes, l'administrateur est responsable du paramétrage global du système. Il gère le cycle de vie des comptes utilisateurs, attribuant les accès et les rôles aux employés. Il est également le garant de la cohérence des données structurelles, telles que le catalogue des chambres et la liste des services proposés. Ce rôle est essentiel pour assurer la maintenance évolutive et la sécurité de l'outil informatique.

## 2.2. Description des scénarios

Afin d'illustrer concrètement le fonctionnement attendu, nous détaillons ici les scénarios nominaux correspondant aux processus métiers clés.

### Scénario 1 : Authentification d'un utilisateur

Le processus d'authentification débute lorsque l'utilisateur accède à l'écran de connexion de l'application. Il est invité à saisir son identifiant unique et son mot de passe confidentiel dans les champs prévus à cet effet. Après validation du formulaire, le système interroge la base de données sécurisée pour vérifier la concordance des informations fournies. En cas de succès, l'utilisateur est redirigé vers le tableau de bord principal, avec une interface adaptée à ses droits spécifiques (réceptionniste ou administrateur). Dans le cas contraire, un message d'erreur discret l'informe de l'échec de la connexion. Si les tentatives infructueuses se répètent au-delà d'un seuil défini, le système déclenche une mesure de sécurité préventive en verrouillant temporairement le compte concerné pour contrer toute tentative d'intrusion par force brute.

### Scénario 2 : Création d'une réservation

La création d'une réservation est initiée par le réceptionniste, généralement suite à une demande client. L'utilisateur commence par renseigner les critères du séjour, notamment les dates d'arrivée et de départ ainsi que le type de chambre souhaité. Le système procède alors à une vérification instantanée des disponibilités, filtrant les chambres libres et aptes à la location sur la période visée. Une fois une chambre sélectionnée, le réceptionniste associe la réservation à un client, soit en le sélectionnant dans la base existante, soit en créant une nouvelle fiche client. Le système offre ensuite la possibilité d'ajouter des services optionnels dès cette étape. Enfin, l'application calcule le montant prévisionnel total et, après validation par l'utilisateur, enregistre la réservation dans la base de données, mettant immédiatement à jour le planning global pour éviter toute surréservation.

### Scénario 3 : Processus de check-in

Lors de l'arrivée physique du client, le réceptionniste active la procédure d'enregistrement (check-in) via l'interface dédiée. Il recherche la réservation concernée et vérifie les informations du client. Une fois l'identité confirmée, l'utilisateur valide l'entrée dans les lieux. Cette action déclenche une mise à jour automatique du statut de la chambre, qui passe de "Réservée" à "Occupée" dans le système. Cette modification d'état est répercutée en temps réel sur le tableau de bord, informant l'ensemble du personnel de la présence du client dans l'établissement.

**Scénario 4 : Génération de facture** Au moment du départ, le processus de facturation est enclenché pour clôturer le séjour. Le système consolide automatiquement l'ensemble des prestations consommées par le client, additionnant le coût des nuitées et celui des divers services ajoutés en cours de séjour. Le réceptionniste peut visualiser un récapitulatif détaillé avant de procéder à l'édition définitive. Sur validation, l'application génère un document PDF officiel, formaté selon l'identité visuelle de l'hôtel, et enregistre la transaction financière. Le statut de la réservation est alors clôturé, et la chambre bascule automatiquement vers un statut transitoire de "Nettoyage", signalant au service d'entretien qu'elle doit être préparée pour le prochain occupant.

### 3. Diagrammes fonctionnels

Pour compléter et formaliser cette description textuelle, l'analyse s'appuie sur des représentations graphiques normalisées issues du langage UML (Unified Modeling Language).

Les diagrammes fonctionnels, et notamment le diagramme de cas d'utilisation, offrent une vision synthétique et structurée des interactions entre les acteurs et le système. Ils permettent de visualiser en un coup d'œil les frontières de l'application, l'ensemble des fonctionnalités offertes, ainsi que les relations de dépendance entre ces fonctionnalités. Ces diagrammes mettent en évidence les mécanismes d'inclusion, illustrant par exemple que l'authentification est un prérequis systématique, et les mécanismes d'extension, montrant comment certaines options viennent enrichir les processus standards.

La modélisation UML détaillée, comprenant non seulement les diagrammes de cas d'utilisation mais également les diagrammes de séquence pour la dynamique des échanges et les diagrammes d'activités pour la logique des processus, sera présentée et commentée de manière exhaustive dans le chapitre 3 consacré à l'analyse et à la conception technique du système.

## III. Étude technique (non fonctionnelle)

Si l'étude fonctionnelle définit ce que le système doit faire, l'étude technique précise comment il doit être construit pour garantir sa viabilité opérationnelle. Cette section détaille les exigences de qualité logicielle et justifie les choix technologiques structurants du projet.

### 1. Besoins non fonctionnels

Les besoins non fonctionnels définissent les contraintes de qualité auxquelles le système doit se conformer pour être utilisable, fiable et pérenne.

#### 1.1. Performance

La performance est un critère déterminant pour l'acceptation de l'outil par les utilisateurs. Le système doit garantir des temps de réponse quasi instantanés pour les opérations courantes telles que la recherche de client ou la vérification de disponibilité, afin de ne pas ralentir le travail du réceptionniste face au client. L'application doit également être capable de supporter la charge liée à de multiples accès simultanés sans dégradation notable du service. Cela implique une optimisation rigoureuse des requêtes vers la base de données et une gestion efficace des ressources système, notamment via l'utilisation d'un pool de connexions pour minimiser la latence des interactions avec le serveur de données.

## **1.2. Sécurité**

Dans un contexte où le système manipule des données personnelles et financières, la sécurité est une exigence absolue. L'authentification doit être robuste, reposant sur des mécanismes cryptographiques éprouvés pour le stockage des mots de passe, tels que le hachage irréversible. Le système doit également intégrer des protections actives contre les attaques courantes, comme les tentatives d'intrusion par force brute ou les injections SQL. La confidentialité des données doit être assurée par une gestion stricte des droits d'accès, garantissant que chaque utilisateur ne peut visualiser ou modifier que les informations nécessaires à sa fonction (principe du moindre privilège).

## **1.3. Fiabilité et disponibilité**

L'application étant critique pour l'exploitation quotidienne de l'hôtel, elle doit faire preuve d'une grande fiabilité. Le système doit être résilient face aux erreurs de saisie ou aux dysfonctionnements mineurs, en proposant des messages d'erreur clairs et en évitant tout arrêt inopiné (crash). L'intégrité des données doit être préservée en toutes circonstances, notamment grâce à la gestion transactionnelle des opérations critiques : une réservation ne doit être confirmée que si toutes les étapes de sa création ont été validées avec succès.

## **1.4. Utilisabilité et ergonomie**

L'ergonomie de l'interface joue un rôle clé dans l'efficacité du personnel. L'application doit proposer une navigation intuitive et une présentation claire de l'information. Les formulaires de saisie doivent être conçus pour guider l'utilisateur et prévenir les erreurs, par exemple en utilisant des sélecteurs de date graphiques plutôt que des champs texte libres. Le système doit fournir un retour visuel immédiat pour chaque action, confirmant le succès d'une opération ou signalant une anomalie de manière explicite, afin que l'utilisateur soit toujours conscient de l'état du système.

## **1.5. Maintenabilité**

Afin de garantir la pérennité du projet et sa capacité à évoluer, le code source doit être maintenable. Cela implique une architecture modulaire favorisant la séparation des responsabilités, le respect des normes de codage et des conventions de nommage, ainsi qu'une documentation technique complète. La conception doit permettre d'ajouter de nouvelles fonctionnalités ou de modifier des règles de gestion existantes avec un impact minimal sur le reste de l'application, facilitant ainsi le travail de maintenance corrective et évolutive.

# **2. Architecture logicielle**

Le choix d'une architecture logicielle adaptée est une décision fondamentale qui conditionne la robustesse et l'évolutivité du système.

## **2.1. Présentation de l'architecture MVC**

Pour répondre aux exigences de maintenabilité et de modularité, nous avons opté pour le patron de conception architectural MVC (Modèle-Vue-Contrôleur). Ce choix est motivé par la nécessité de séparer clairement les trois aspects fondamentaux de l'application : la gestion des données, l'interface utilisateur et la logique de contrôle. Cette séparation des préoccupations permet de réduire la complexité du développement, facilite les tests unitaires et permet de faire évoluer l'interface graphique indépendamment du cœur métier.

Dans notre implémentation, le Modèle encapsule les données de l'hôtel et les règles métier associées, garantissant leur intégrité. La Vue est responsable de la présentation visuelle de ces informations à l'utilisateur, sans contenir de logique complexe. Le Contrôleur agit comme un chef d'orchestre,

interceptant les actions de l'utilisateur sur la Vue pour déclencher les traitements appropriés dans le Modèle, puis mettant à jour l'affichage en conséquence. Cette architecture structurée est particulièrement adaptée à la complexité d'un système de gestion hôtelière, où les mêmes données (comme l'état d'une chambre) doivent être manipulées et affichées dans de multiples contextes.

### **3. Choix des outils et technologies**

La sélection de la stack technologique a été guidée par une volonté de cohérence, de performance et d'adéquation avec les objectifs pédagogiques du projet.

#### **3.1. Langage de programmation : Java 25.0.1**

Java s'impose comme le langage de référence pour ce projet académique. Langage purement orienté objet, il offre la rigueur structurelle nécessaire pour modéliser des systèmes complexes. Sa portabilité ("Write Once, Run Anywhere") garantit que l'application pourra être déployée sur différents environnements sans recompilation. De plus, Java bénéficie d'un écosystème riche et mature, fournissant une multitude de bibliothèques standards pour la gestion des bases de données, des interfaces graphiques et de la sécurité. Le choix de la version 25 assure l'accès aux fonctionnalités les plus récentes et aux optimisations de performance de la machine virtuelle Java (JVM). Face à des alternatives comme C++ (plus complexe) ou Python (moins adapté aux applications de bureau lourdes), Java représente le meilleur compromis entre performance, robustesse et facilité de développement.

#### **3.2. Interface graphique : JavaFX 17**

Pour la réalisation de l'interface utilisateur, notre choix s'est porté sur JavaFX. Cette technologie moderne succède à Swing en proposant une approche plus flexible et performante de la création d'IHM. JavaFX permet de séparer la description de l'interface (via le langage FXML) de la logique applicative, et offre des capacités de stylisation avancées grâce aux feuilles de style CSS. Cette séparation facilite grandement le travail de design et permet de créer des interfaces visuellement attractives et ergonomiques. Contrairement aux interfaces web qui nécessiteraient une architecture plus complexe, JavaFX est parfaitement intégré à l'écosystème Java, simplifiant le développement d'une application de bureau riche et réactive.

#### **3.3. Base de données : MySQL 8.0**

La persistance des données est confiée au SGBD relationnel MySQL. Ce choix est justifié par la nature fortement structurée des données hôtelières (clients, réservations, chambres), qui s'inscrit naturellement dans un modèle relationnel. MySQL est reconnu pour sa fiabilité, ses performances élevées en lecture/écriture et sa conformité aux propriétés ACID, indispensables pour garantir l'intégrité des transactions financières et des réservations. Open-source et largement documenté, il s'intègre nativement avec Java via le pilote JDBC, offrant une solution robuste et éprouvée pour la gestion des données du projet.

#### **3.4. Pool de connexions : HikariCP**

Afin d'optimiser les performances d'accès à la base de données, nous avons intégré la bibliothèque HikariCP. Dans une application traditionnelle, l'ouverture et la fermeture répétées de connexions à la base de données sont des opérations coûteuses en ressources et en temps. HikariCP résout ce problème en maintenant un "pool" de connexions ouvertes et prêtes à l'emploi. Cette approche réduit drastiquement la latence des requêtes et améliore la réactivité globale de l'application, en particulier lorsque plusieurs utilisateurs interagissent simultanément avec le système.



### 3.5. Architecture : Design Patterns

Au-delà du pattern MVC, l'application intègre plusieurs patrons de conception pour répondre à des problèmes récurrents de manière élégante et standardisée. Le pattern **Singleton** est utilisé pour garantir l'unicité du gestionnaire de connexion à la base de données, optimisant ainsi l'utilisation des ressources. Le pattern **DAO (Data Access Object)** est mis en œuvre pour abstraire la couche de persistance, découplant la logique métier des détails techniques des requêtes SQL. Le pattern **Factory** facilite la création d'objets complexes, tandis que le pattern **Observer** permet de mettre à jour l'interface en temps réel lors de modifications des données. Enfin, le pattern **Strategy** offre la flexibilité nécessaire pour implémenter différentes règles de calcul tarifaire sans figer le code.

### 3.6. Génération PDF : iText

Pour répondre au besoin critique de génération de factures, la bibliothèque iText a été retenue. C'est une solution standard de l'industrie pour la création programmatique de documents PDF. Elle permet de transformer les données dynamiques de l'application (détails de la réservation, prix) en documents professionnels, formatés avec précision et non modifiables, répondant ainsi aux exigences légales et administratives de la facturation.

### 3.7. Outils de développement

L'environnement de développement repose sur l'IDE IntelliJ IDEA, choisi pour son intelligence d'assistance au codage, ses outils de débogage avancés et son excellente intégration de l'écosystème Java. La gestion de versions et la collaboration sont assurées par Git et la plateforme GitHub. Ces outils sont indispensables pour suivre l'historique des modifications, gérer les branches de développement parallèle et fusionner les travaux des différents membres de l'équipe, assurant ainsi la cohérence du projet tout au long de son cycle de vie.

### 3.8. Justification des choix

L'ensemble des choix technologiques et architecturaux opérés forme un écosystème cohérent, spécifiquement taillé pour répondre aux objectifs du projet. L'alliance de Java et de l'architecture MVC assure la robustesse et la maintenabilité requises par les exigences académiques. L'utilisation de JavaFX et de MySQL répond aux besoins fonctionnels d'ergonomie et de fiabilité des données. L'intégration de composants de performance comme HikariCP et de bibliothèques spécialisées comme iText démontre une volonté de produire une solution de qualité professionnelle, capable de gérer les contraintes réelles d'une exploitation hôtelière. Bien que le choix d'une application de bureau limite l'accessibilité à distance par rapport à une solution web, il a permis de concentrer les efforts sur la richesse fonctionnelle et la maîtrise des concepts objets, respectant ainsi parfaitement le cadre pédagogique et les délais impartis de quatre semaines.

## IV. Conclusion

Ce chapitre a permis d'établir une cartographie précise des besoins du projet. Nous avons traduit la problématique initiale en un cahier des charges fonctionnel détaillé, structurant le système autour de modules clés tels que la gestion des réservations et la facturation. Parallèlement, l'étude technique a posé les jalons d'une architecture logicielle solide, justifiant l'emploi de technologies éprouvées et de patrons de conception standards pour garantir la sécurité, la performance et la maintenabilité de la solution.

Ces exigences fonctionnelles et techniques, clairement établies et argumentées, constituent le fondement inaltérable sur lequel repose la conception détaillée. Elles serviront de guide de référence pour le chapitre suivant, consacré à la modélisation formelle du système à travers les diagrammes UML et la conception de la base de données.

# **Chapitre 3 :**

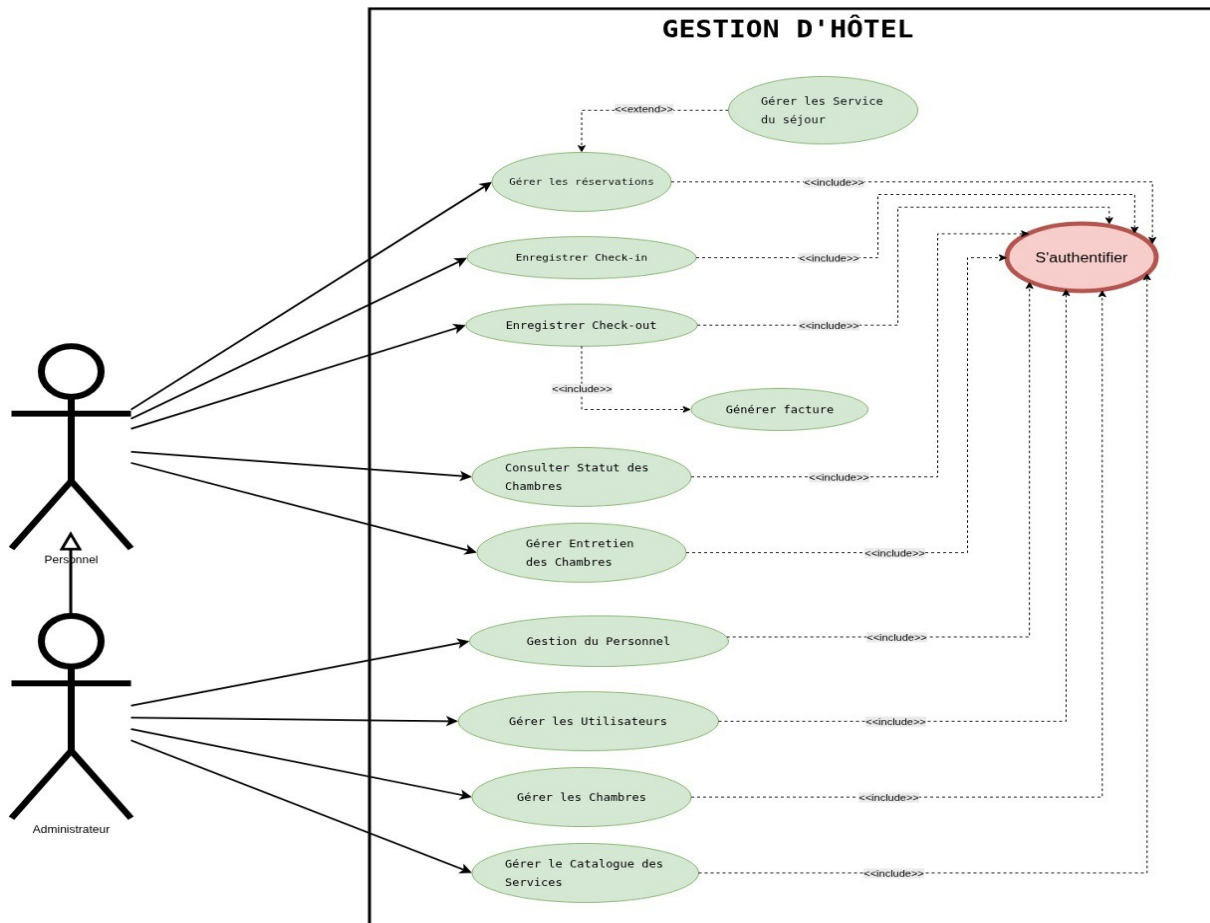
## **Analyse et Conception**

## I. Introduction

Ce chapitre constitue la charnière entre l'expression des besoins et la réalisation technique. Il décrit comment le système a été pensé et modélisé avant d'être codé, en passant du besoins utilisateur à la spécification technique.

## II. Analyse UML

### 1. Diagramme cas d'utilisation



#### Description :

Le diagramme présenté est un diagramme de cas d'utilisation UML qui décrit les fonctionnalités principales du système de gestion hôtelière du point de vue des acteurs externes. Il met en évidence les interactions entre les utilisateurs et le système, ainsi que les relations entre les différents cas d'utilisation.

#### Acteurs principaux :

Personnel Représente les employés opérationnels de l'hôtel (réceptionnistes, personnel d'entretien, etc.).

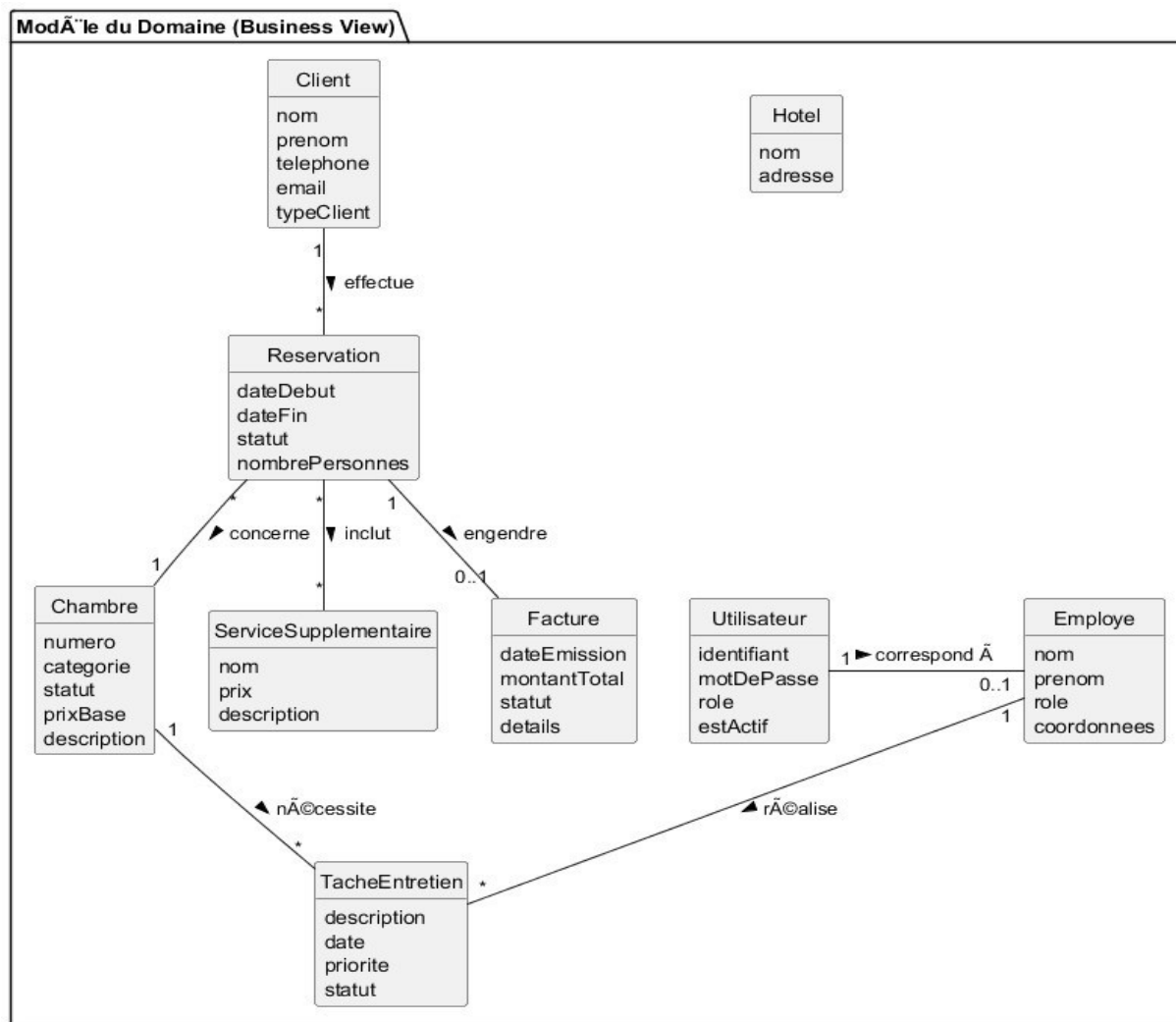
Administrateur Représente l'utilisateur disposant de privilèges élevés pour la configuration et la gestion avancée du système.

#### Relations entre les cas d'utilisation :

- <<include>> (inclusion obligatoire) : La plupart des cas d'utilisation fonctionnels nécessitent que l'utilisateur soit authentifié.
- <<extend>> (extension conditionnelle) : Gérer les Services du séjour est une extension optionnelle de Gérer les réservations (le personnel peut ajouter des services pendant le séjour).

Ce diagramme de cas d'utilisation fournit une vue claire des fonctionnalités offertes par le système et des droits d'accès selon le rôle de l'utilisateur. Il met particulièrement l'accent sur la sécurité (authentification obligatoire pour toutes les actions sensibles) et sur la séparation des responsabilités entre le personnel opérationnel et l'administrateur. Il constitue une base essentielle pour la définition des exigences fonctionnelles et pour la conception des interfaces utilisateur du projet.

## 2. Diagramme de classes



### Description :

Le diagramme de classes présenté constitue le pivot de la modélisation orientée objet du système. Il décrit la structure statique du domaine métier en identifiant les principales entités, leurs attributs et les relations entre elles.

### Classes principales et leurs attributs :

**Client :** Représente une personne effectuant une réservation.

- Attributs : nom, prénom, téléphone, email, typeClient.

**Hotel :** Représente l'établissement hôtelier lui-même.

- Attributs : nom, adresse.

**Chambre :** Entité centrale représentant une unité d'hébergement.

- Attributs : numero, categorie, statut, prixBase, description.

**Reservation :** Classe pivot liant un client à une chambre pour une période donnée. Elle constitue une classe d'association enrichie.

- Attributs : dateDebut, dateFin, statut, nombrePersonnes.

**ServiceSupplementaire** : Représente les services additionnels proposés (ex. : petit-déjeuner, spa, etc.).

- Attributs : nom, prix, description.

**Facture** : Document financier généré à l'issue d'une réservation.

- Attributs : dateEmission, montantTotal, statut, details.

**Utilisateur** : Représente les personnes accédant au système informatique (employés).

- Attributs : identifiant, motDePasse, role, estActif.

**Employe** : Décrit les employés de l'hôtel.

- Attributs : nom, prenom, role, coordonnees.

**TacheEntretien** : Gère les opérations de maintenance et de nettoyage des chambres.

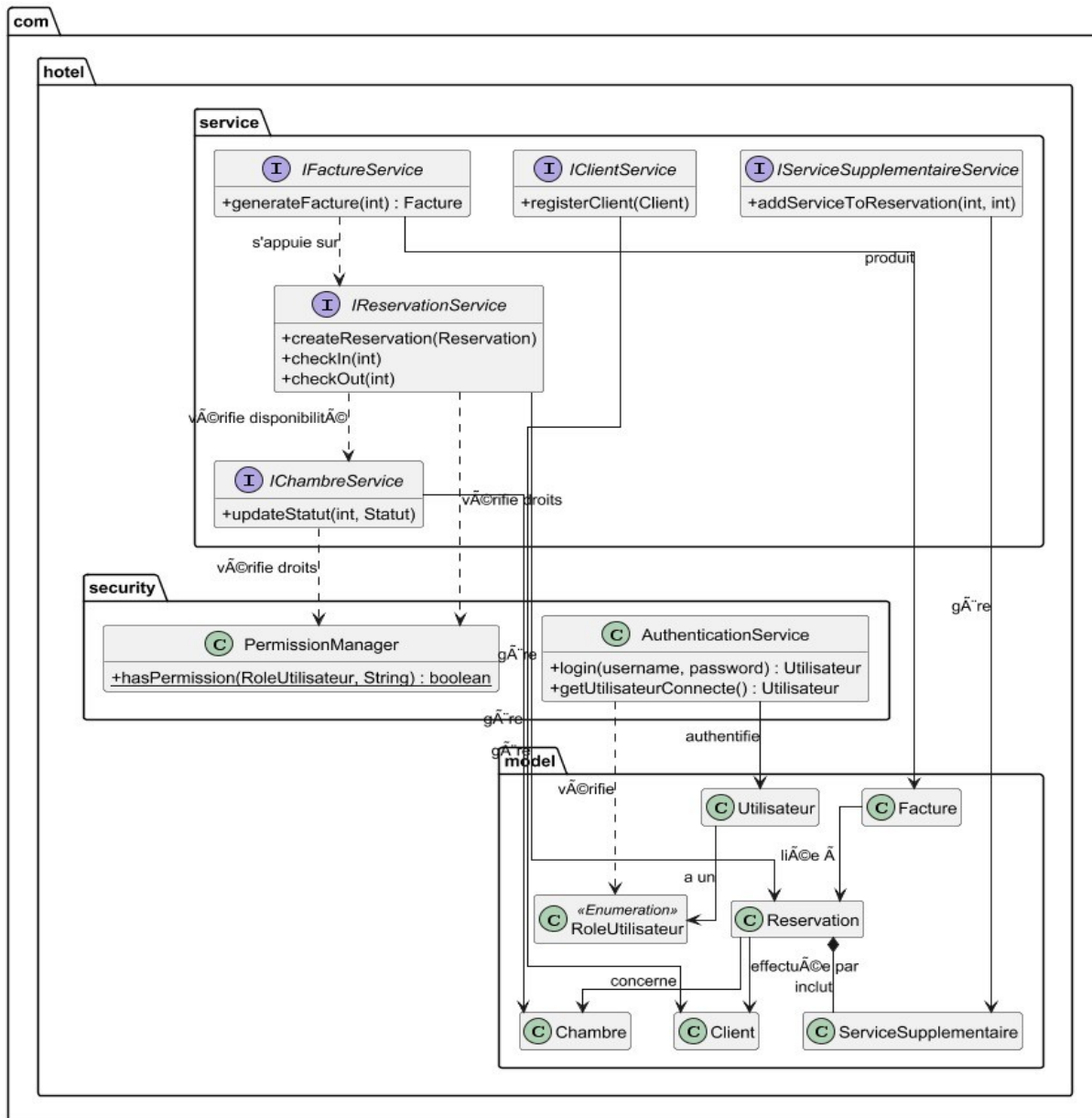
- Attributs : description, date, priorite, statut.

**Relations principales :**

- Un Client effectue une ou plusieurs Reservations (relation 1..\*).
- Une Reservation concerne exactement une Chambre et peut inclure zéro ou plusieurs ServicesSupplementaires.
- Une Reservation engendre exactement une Facture.
- Une Facture est réalisée par un Utilisateur.
- Un Utilisateur correspond à zéro ou un Employe (un employé peut avoir un compte système, mais pas obligatoirement).
- Une Chambre peut nécessiter zéro ou plusieurs TachesEntretien.

Ce diagramme de classes offre une vue claire et complète de la structure statique du domaine, en mettant en évidence les concepts clés (client, réservation, chambre, facturation, entretien) et leurs interrelations. Il sert de fondation pour la conception détaillée, la génération du modèle relationnel de la base de données et l'implémentation des objets métier dans le système.

### 3. diagramme de packages



#### Description :

Le diagramme présenté illustre **l'architecture logicielle** du système, organisée selon une structure en packages et respectant les principes de séparation des responsabilités (notamment une approche en couches avec des éléments de conception orientée services). Il s'agit d'un diagramme de packages et de classes UML montrant les composants principaux et leurs interactions.

#### Packages principaux :

**com.hotel** Contient les packages suivants :

- **service** : Regroupe les interfaces des services métier.
- **security** : Gère l'authentification et la gestion des permissions.
- **model** : Contient les classes du domaine (entités métier).
- **gère** : Indique les dépendances de gestion.

## **Interfaces de service (dans le package service) :**

### ➤ **IReservationService :**

Méthodes : createReservation(Reservation), checkIn(int), checkOut(int). Gère le cycle de vie des réservations.

### ➤ **ICambreService :**

Méthode : updateStatut(int, Statut). Responsable de la mise à jour du statut des chambres et de la vérification de disponibilité.

### ➤ **IFactureService :**

Méthode : generateFacture(int) : Facture. S'appuie sur les données de réservation pour produire la facture.

### ➤ **IClientService :**

Méthode : registerClient(Client). Permet l'enregistrement des clients.

### ➤ **IServiceSupplementaireService :**

Méthode : addServiceToReservation(int, int). Ajoute des services supplémentaires à une réservation.

## **Composants de sécurité (dans le package security) :**

### ➤ **PermissionManager :**

Méthode : hasPermission(RoleUtilisateur, String) : boolean. Vérifie les droits d'accès en fonction du rôle de l'utilisateur.

### ➤ **AuthenticationService :**

Méthodes : login(username, password) : Utilisateur, getUtilisateurConnecte() : Utilisateur. Authentifie les utilisateurs et récupère l'utilisateur connecté.

## **Package model : Classes du domaine**

- Utilisateur → lié à RoleUtilisateur (énumération).
- Reservation → effectuée par Client, concerne Chambre, inclut ServiceSupplementaire, liée à Facture.
- Relations principales :
  - ✓ Un Client effectue des Reservations.
  - ✓ Une Reservation concerne une Chambre et peut inclure plusieurs ServiceSupplementaire.
  - ✓ Une Reservation génère une Facture.

## Dépendances et flux

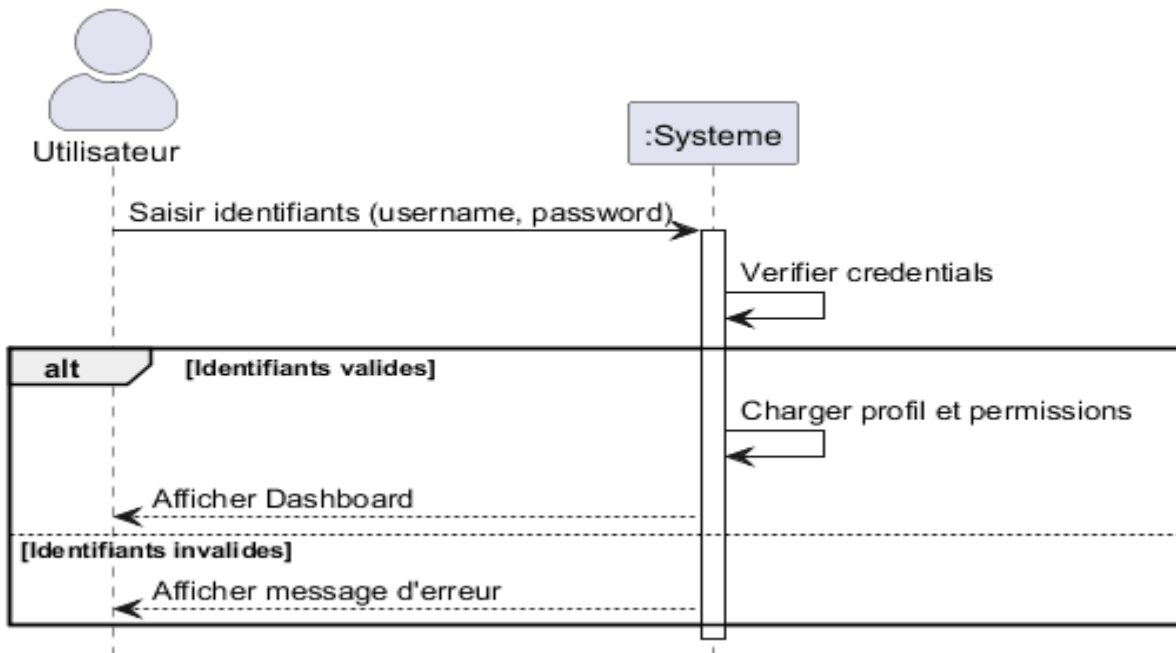
- Les services métier dépendent des entités du modèle et vérifient les droits via PermissionManager.
- IReservationService et ICambreService vérifient la disponibilité et les droits avant toute opération.
- AuthenticationService authentifie les appels vers les services sensibles.

Ce diagramme met en évidence une architecture propre, avec une claire séparation entre la couche de service (logique métier), la couche de sécurité (contrôle d'accès) et la couche modèle (données persistantes). Il facilite la maintenabilité, la sécurité et l'extensibilité du système en isolant les responsabilités et en centralisant la gestion des autorisations et de l'authentification. Ce modèle constitue une base solide pour l'implémentation technique du projet.

## 4. Diagrammes de séquence

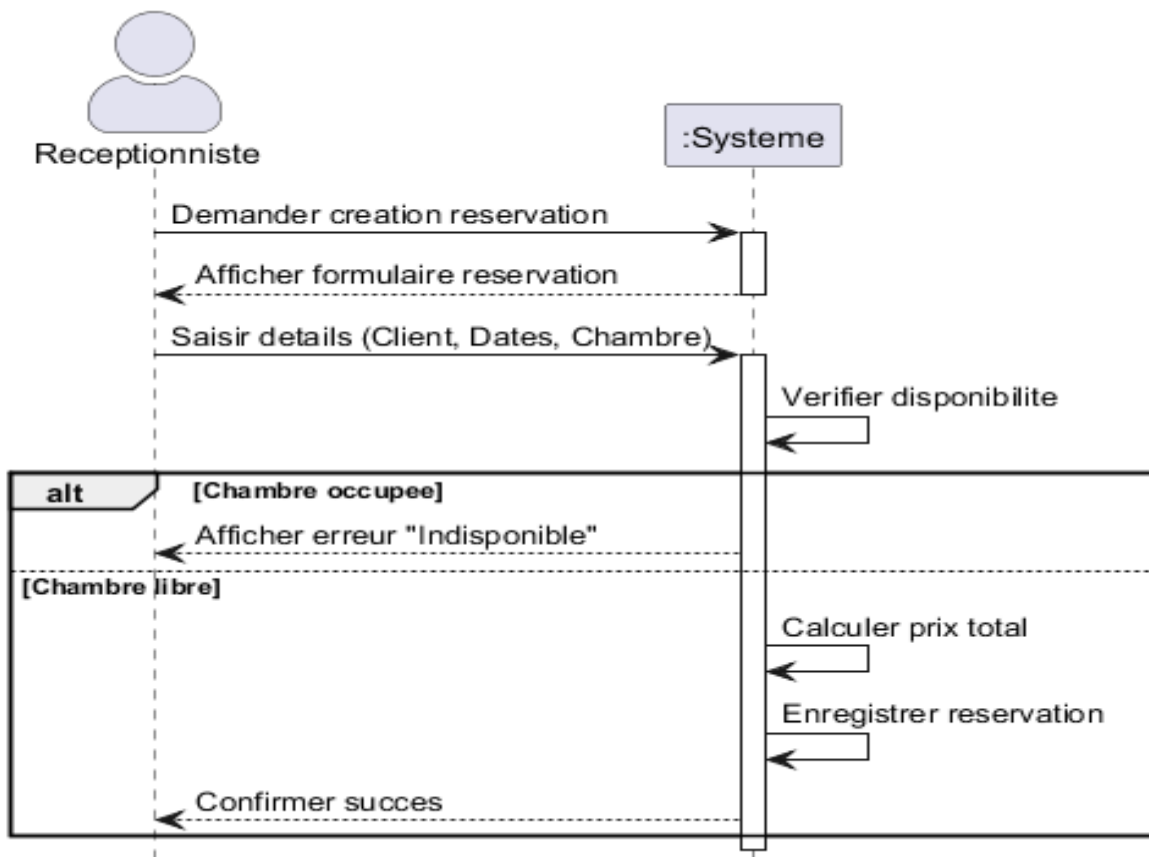
### 4.1. Diagrammes de séquence Boîte Noire

#### Diagramme : Authentification



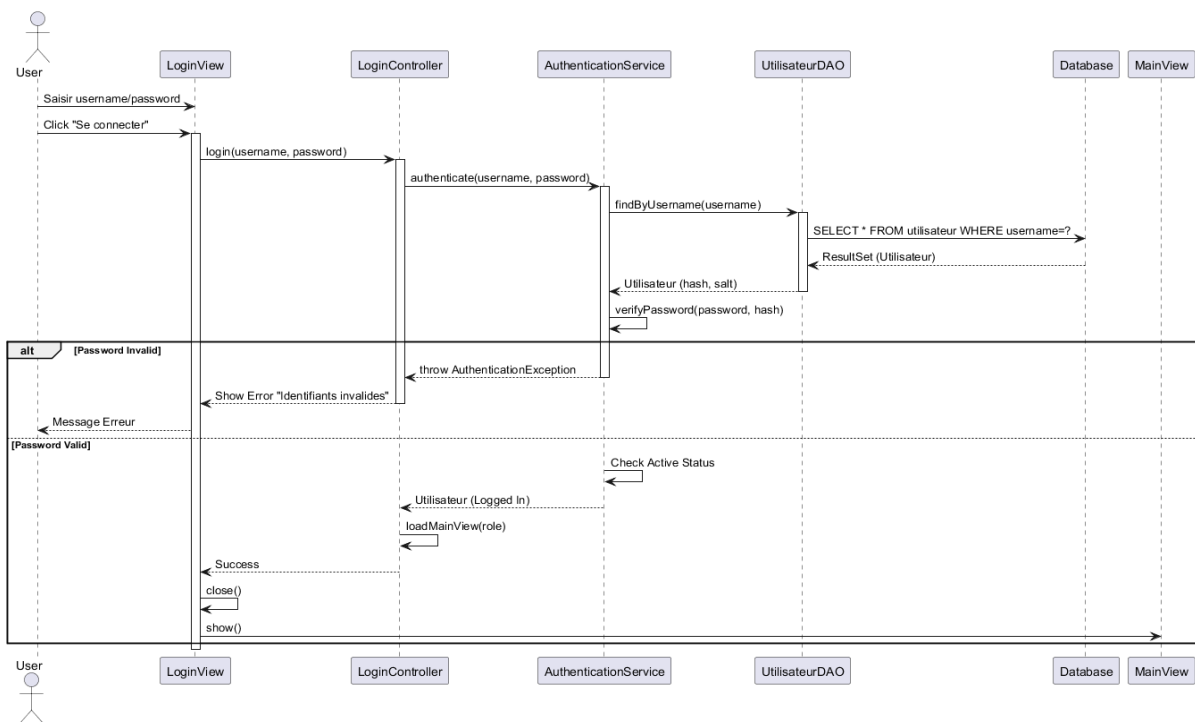


## 4.2. Diagramme : Création d'une réservation



## 5. Diagrammes de séquence Boîte Blanche

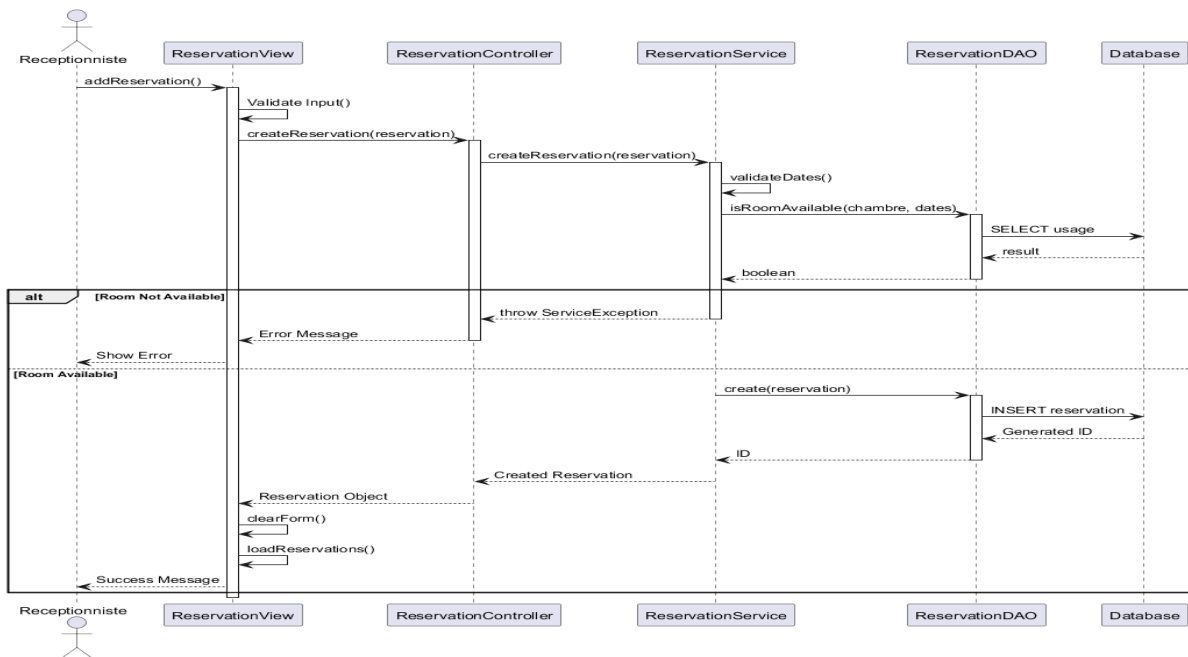
### 5.1. Diagramme de Séquence : Authentification



## Description :

- Saisie : L'utilisateur soumet ses identifiants.
- Vérification : AuthenticationService prend le relais.
- Persistance : Le service récupère l'utilisateur en base via UtilisateurDAO (SELECT\*).
- Sécurité : Le service compare le mot de passe saisi avec le hash stocké (verifyPassword).
- Cas Valide : L'utilisateur est authentifié, le Dashboard s'ouvre.
- Cas Invalide : Une exception est levée (AuthException) et un message d'erreur est affiché.

## 5.2. Diagramme de Séquence : Création de Réservation



## Description :

Ce diagramme montre le processus de réservation avec validation.

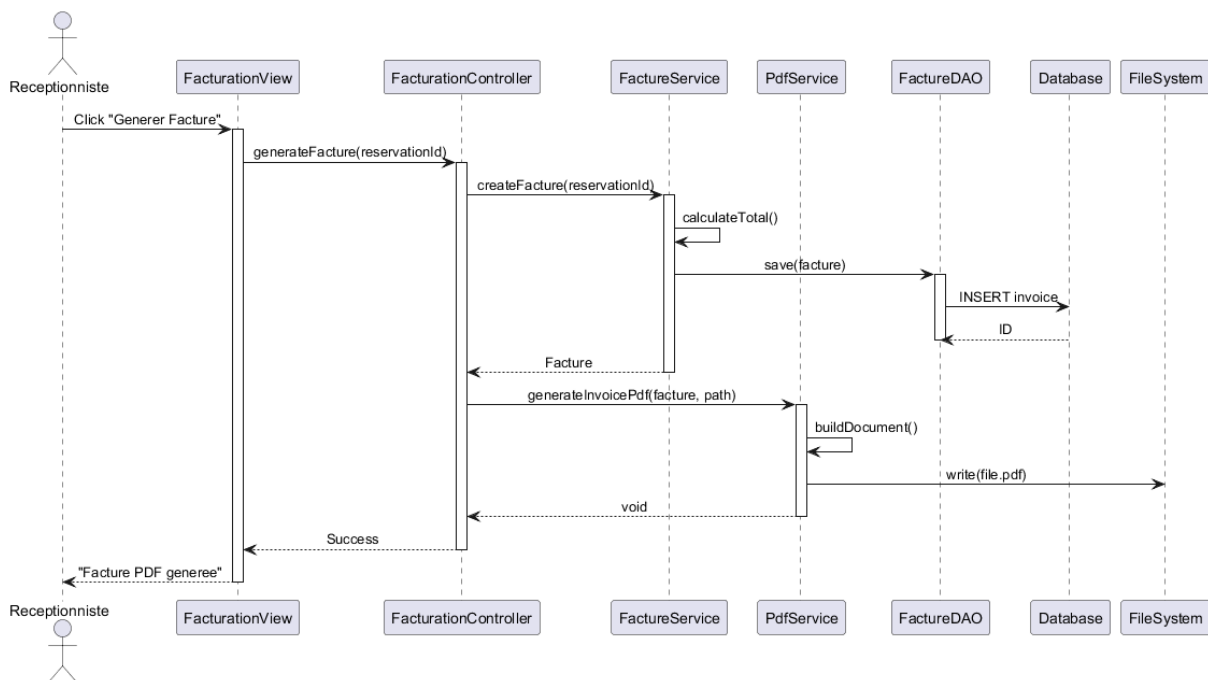
## Scénario :

- Soumission : Le réceptionniste envoie le formulaire de réservation.
- Traitement Métier : ReservationController appelle ReservationService.
- Le service vérifie la disponibilité de la chambre pour les dates données (isAvailable).
- Cette vérification implique une requête en base pour détecter les chevauchements (Check overlap).

## Résultat :

- Si Disponible : La réservation est insérée en base (INSERT), l'ID est généré, et la vue est mise à jour.
- Si Occupée : Le processus s'arrête, une exception est levée et le message "Room Unavailable" est affiché.

### 5.3. Diagramme de Séquence : Génération de Facture PDF



#### Description :

Ce diagramme illustre le processus complet de création et d'export d'une facture.

#### Scénario :

- Déclenchement : Le réceptionniste clique sur "Générer Facture".
- Orchestration : FacturationController demande au service de créer la facture.

#### Logique Métier (FactureService) :

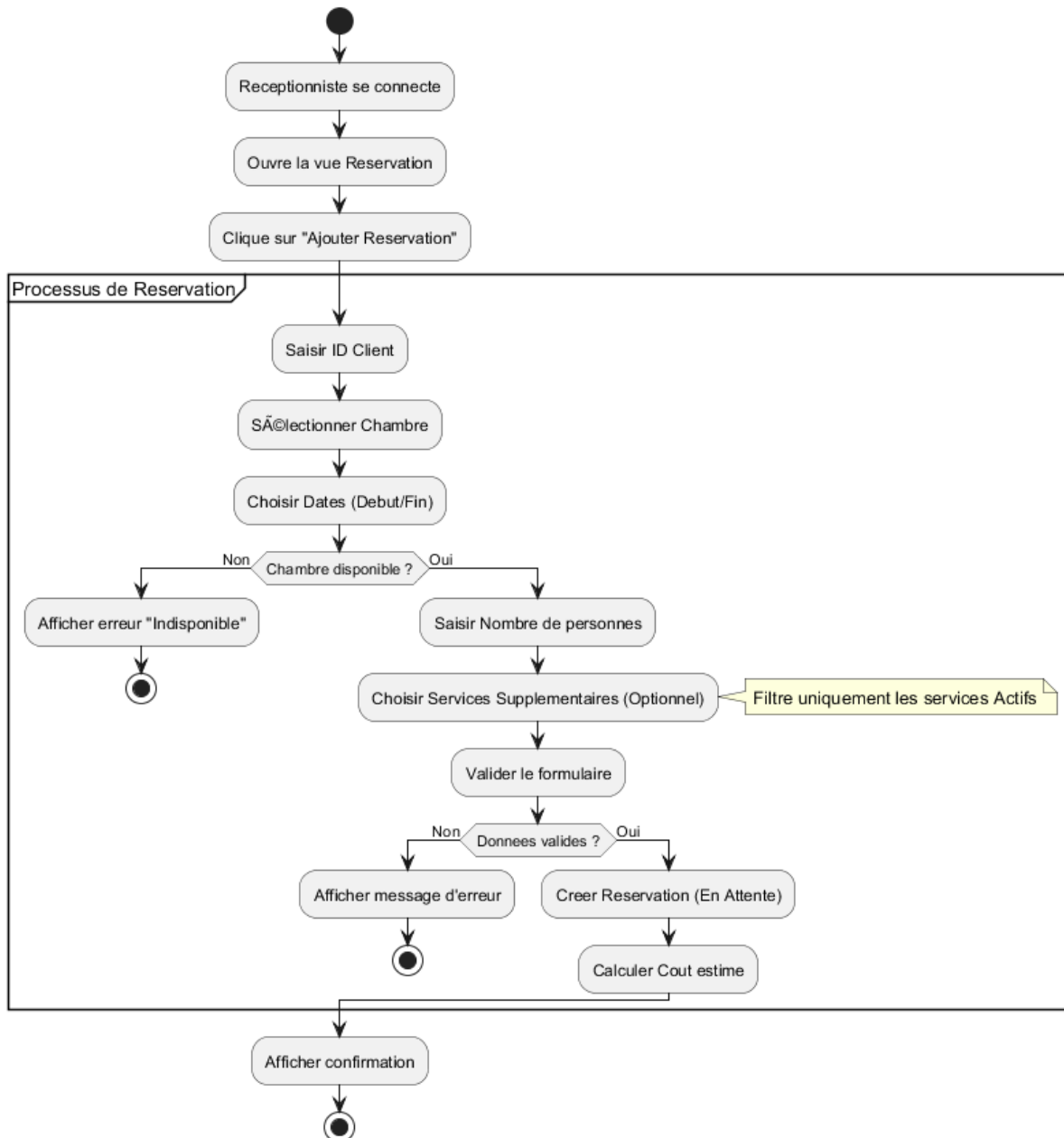
- Il calcule le montant total (calculateTotal).
- Il sauvegarde l'objet facture en base via FactureDAO (INSERT).
- Il appelle le PdfService pour la génération du document physique.

#### Génération de Fichier :

- PdfService construit le document (mise en page, tableau).
- Il écrit le fichier .pdf sur le système de fichiers (FileSystem).

**Fin** : Un message de succès confirme que le PDF a été généré.

## 6. Diagrammes d'activités



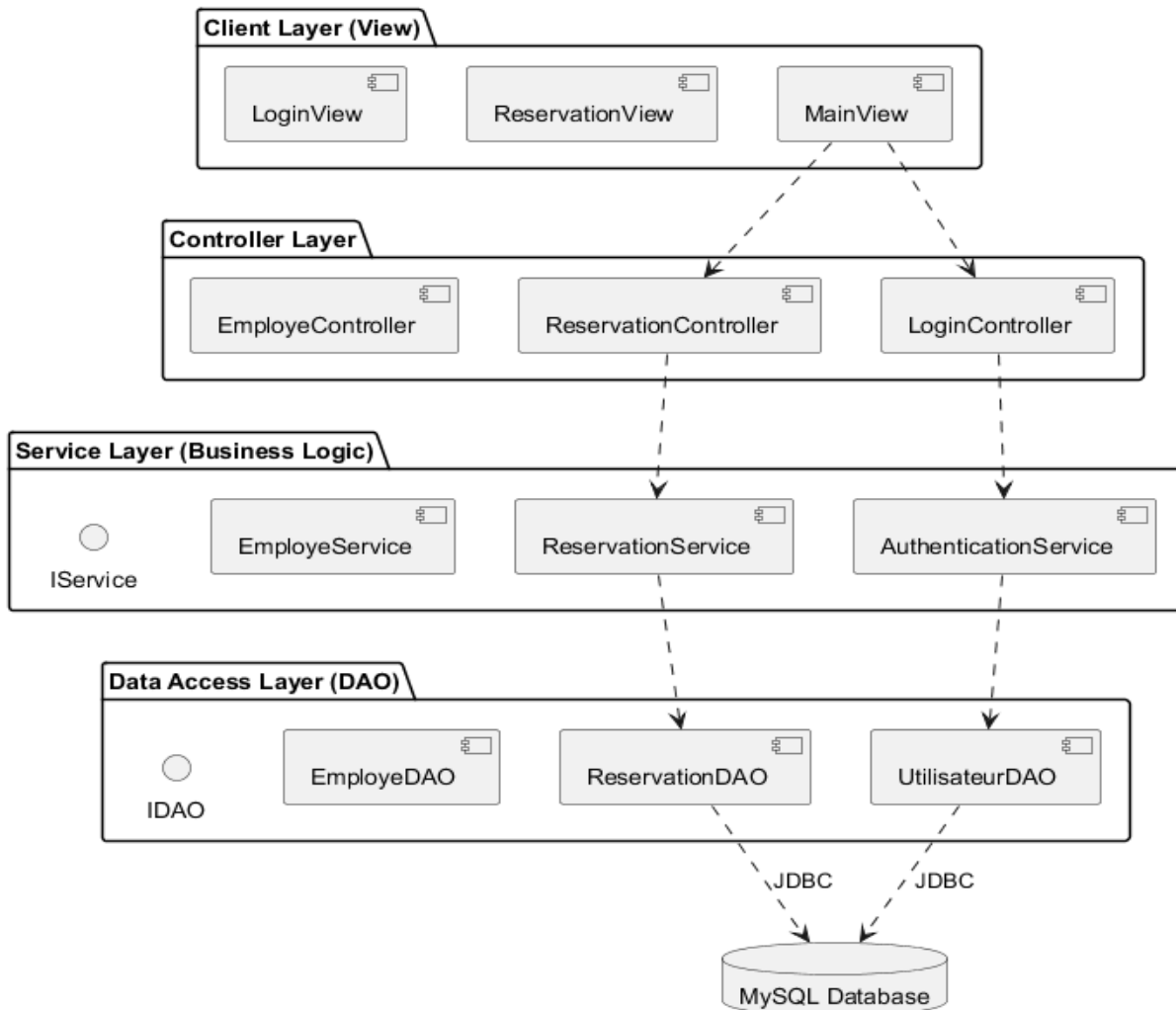
### Description :

Ce diagramme d'activité UML décrit succinctement le flux de création d'une réservation par un réceptionniste :

- Le réceptionniste se connecte, ouvre la vue Réservation et clique sur « Ajouter Réservation ».
- Il saisit l'ID client, sélectionne une chambre et choisit les dates de séjour.
- Le système vérifie la disponibilité : si la chambre est indisponible → erreur et fin ; sinon → continuation.
- Saisie du nombre de personnes et choix optionnel de services supplémentaires (filtrés sur les services actifs uniquement).
- Validation du formulaire : si données invalides → message d'erreur ; sinon → création de la réservation (statut « En Attente »).
- Calcul du coût estimé et affichage de la confirmation.

Ce diagramme illustre le processus principal de réservation avec gestion des erreurs (indisponibilité, saisie incorrecte) et des chemins alternatifs, en mettant l'accent sur la simplicité et la robustesse du flux utilisateur.

## 7. Diagramme de composants



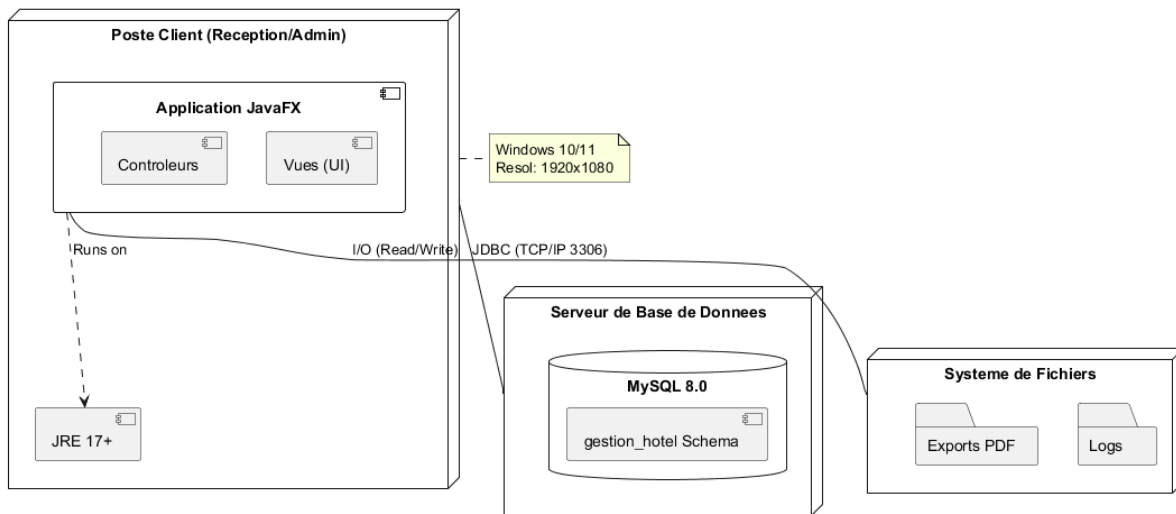
### Description :

Ce diagramme présente l'architecture logicielle en couches du système :

- Client Layer (View) : Interfaces JavaFX (`LoginView`, `ReservationView`, `MainView`).
- Controller Layer : Contrôleurs (`EmployeeController`, `ReservationController`, `LoginController`).
- Service Layer : Logique métier (`EmployeeService`, `ReservationService`, `AuthenticationService`).
- Data Access Layer (DAO) : Accès aux données (`EmployeeDAO`, `ReservationDAO`, `UtilisateurDAO`).
- MySQL Database : Persistance via JDBC.

Les dépendances vont du haut vers le bas (vues → contrôleurs → services → DAO → BD), assurant une séparation claire des responsabilités, une bonne maintenabilité et une évolution facilitée de l'application.

## 8. Diagramme de Déploiement



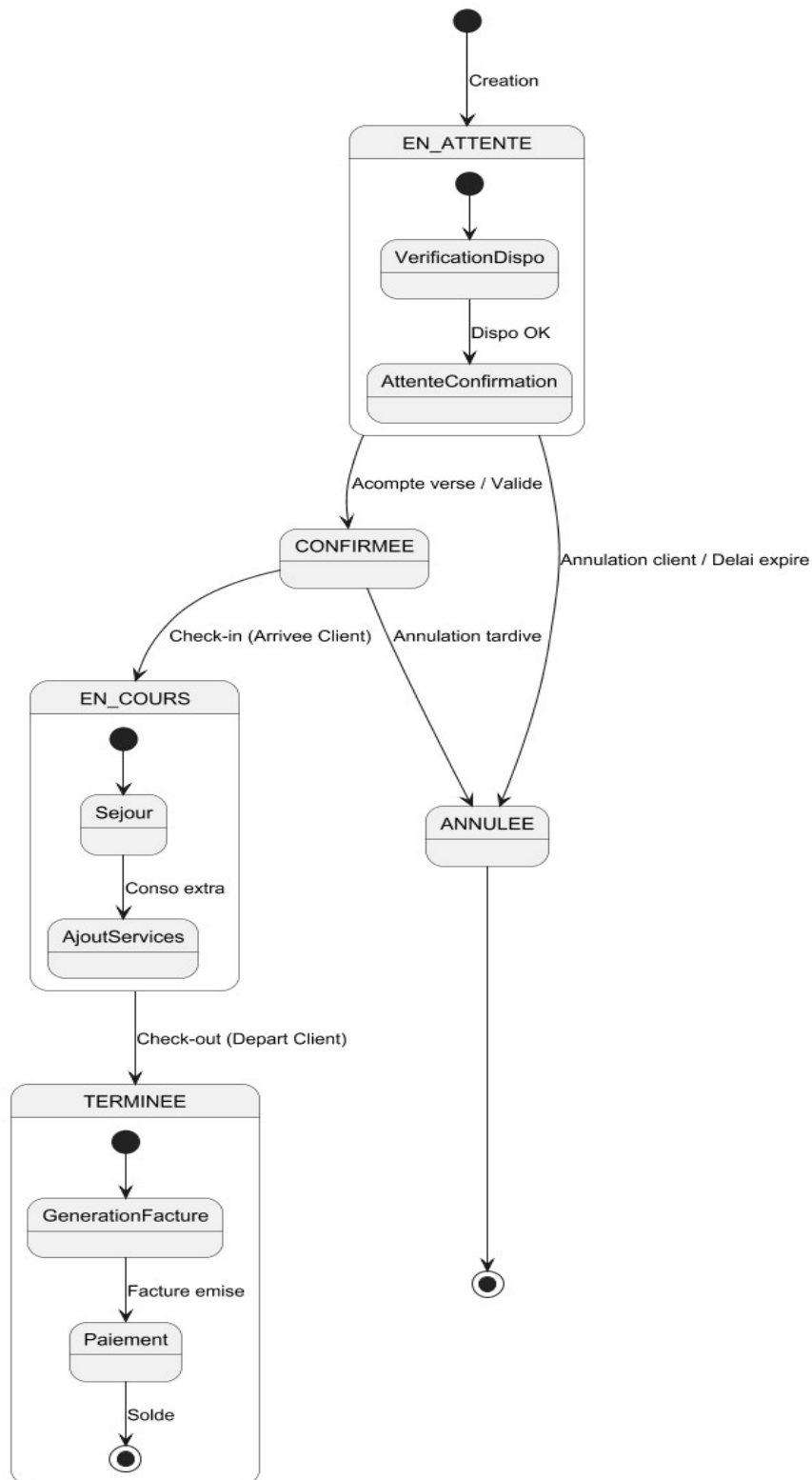
### Description :

Ce diagramme de déploiement présente l'architecture physique du système :

- Poste Client (Réception/Admin) : Ordinateur sous Windows 10/11 (résolution 1920x1080), exécutant l'application JavaFX (avec contrôleurs et vues UI) sur JRE 17+.
- Serveur de Base de Données : Héberge MySQL 8.0 avec le schéma gestion\_hotel. Communication via JDBC (TCP/IP port 3306) pour les opérations de lecture/écriture.
- Système de Fichiers : Stocke les exports PDF (factures, etc.) et les logs de l'application.

Il illustre une architecture client-serveur simple et robuste, adaptée au déploiement dans un environnement hôtelier.

## 9. Diagramme d'États-Transitions – Cycle de Vie d'une Réservation



### Description :

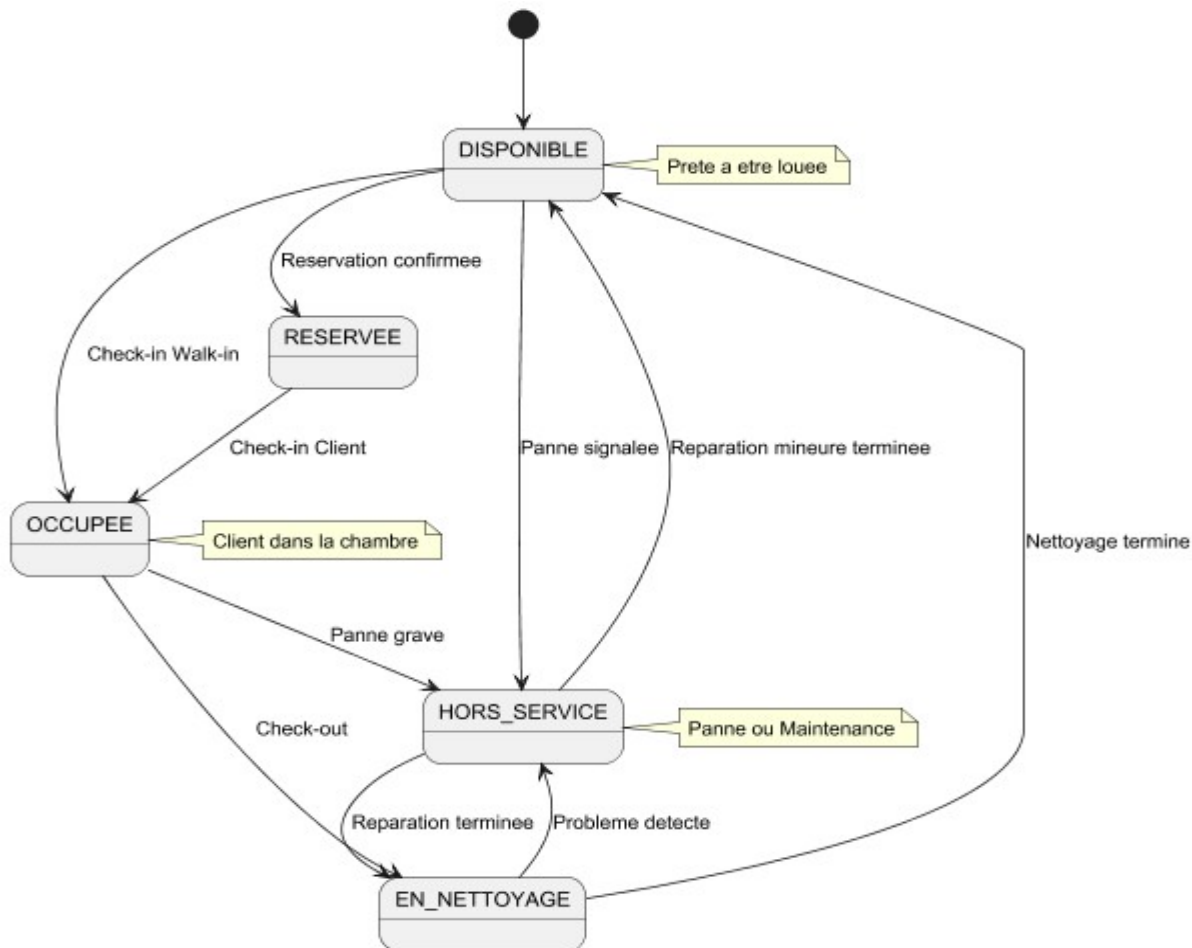
Ce diagramme modélise le cycle de vie d'une réservation à travers ses états successifs :

- **EN\_ATTENTE** → Vérification disponibilité → Attente confirmation.
- **CONFIRMEE** → Par acompte ou validation.
- **EN\_COURS** → Après check-in ; possibilité d'ajouter services/consos extra.

- TERMINEE → Après check-out ; génération facture, paiement et solde.
- ANNULEE → En cas d'annulation (client, délai expiré ou tardive).

Il définit clairement les transitions et règles de passage d'un état à l'autre, servant de référence pour la gestion des statuts dans l'application.

## 10. Diagramme d'États-Transitions-cycle de vie d'une chambre



### Description :

Ce diagramme décrit le cycle de vie d'une chambre à travers ses états :

- DISPONIBLE → Prête à être louée.
- RÉSERVÉE → Après réservation confirmée ou check-in walk-in.
- OCCUPÉE → Client dans la chambre.
- EN\_NETTOYAGE → Après check-out ; nettoyage en cours.
- HORS\_SERVICE → En cas de panne grave ou problème détecté.

Transitions principales :

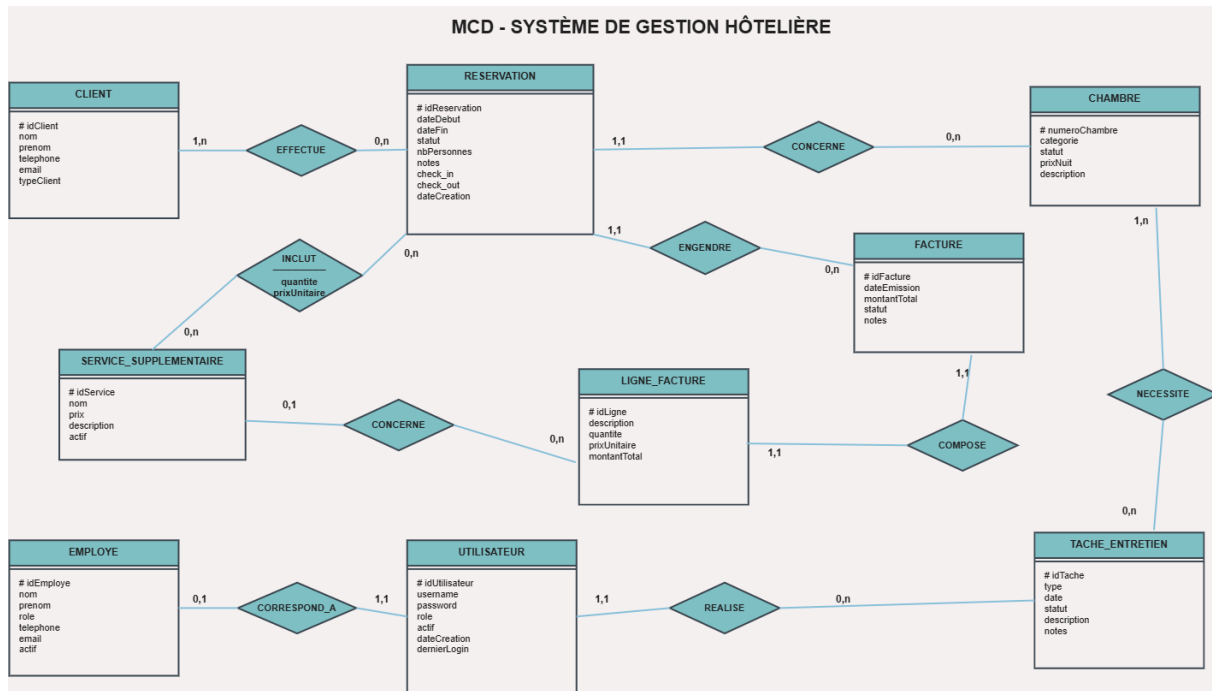
- Réservation/Check-in → OCCUPÉE
- Check-out → EN\_NETTOYAGE
- Nettoyage/Réparation terminé → DISPONIBLE
- Panne signalée → HORS\_SERVICE

Il assure un suivi précis du statut des chambres pour optimiser la disponibilité et la maintenance.

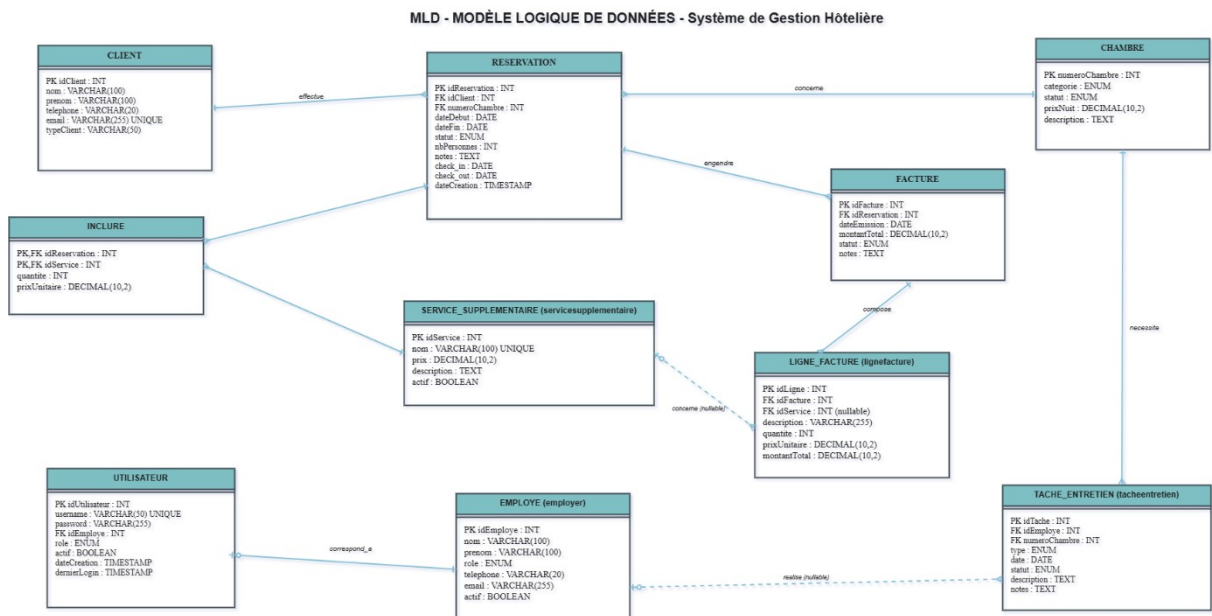


## 11. Modèle de données

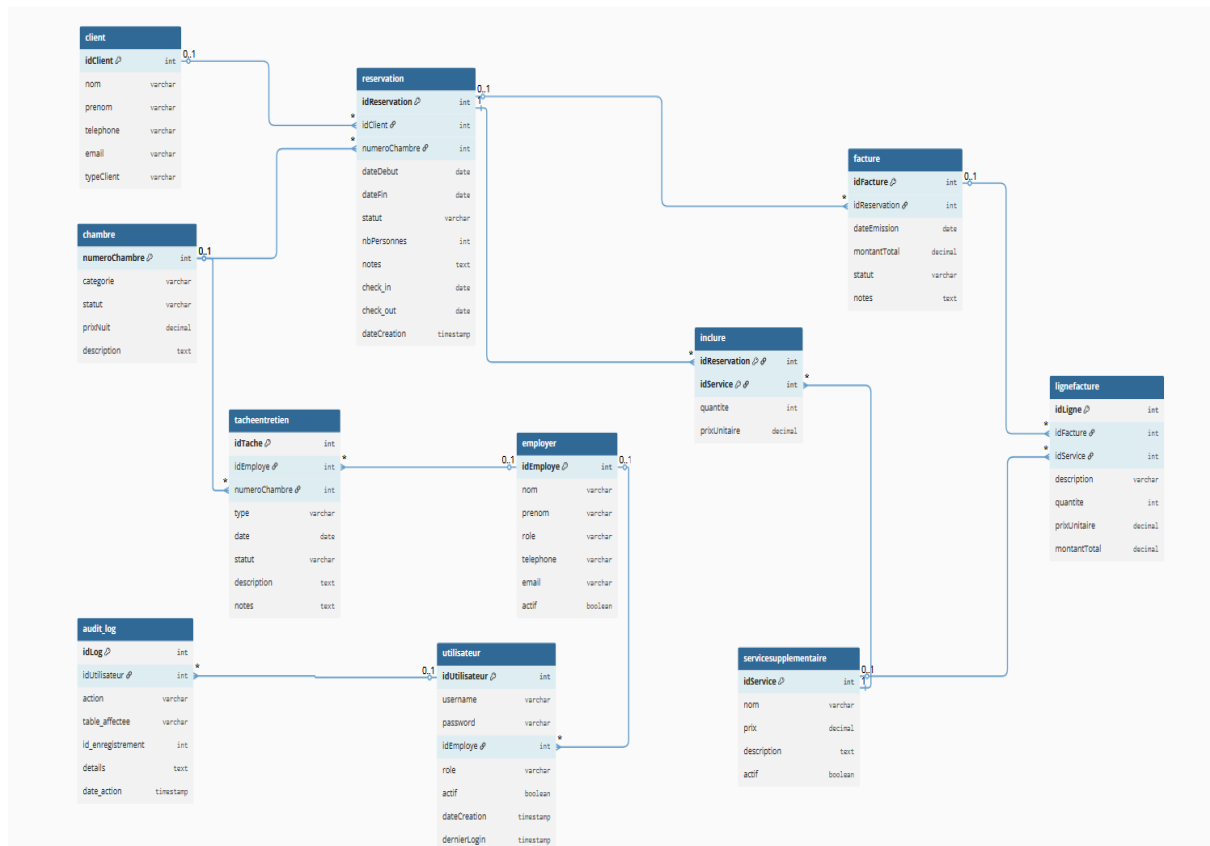
### 11.1. 3.2.1. Modèle Conceptuel de Données (MCD)



### 11.2. Modèle Logique de Données (MLD)



## 11.3. Modèle Physique de Données (MPD)



## 12. Conception de l'architecture

### 12.1. Architecture logicielle

L'application suit le pattern **MVC (Modèle-Vue-Contrôleur)**:

- **Modèle** : Gestion des données et règles métier (DAO, Beans).
- **Vue** : Interface graphique javafx présentée à l'utilisateur.
- **Contrôleur** : Chef d'orchestre qui reçoit les entrées utilisateur et met à jour le modèle/vue.

### 12.2. Description des modules

L'application est découpée en modules fonctionnels indépendants, chacun encapsulant sa propre logique métier :

#### 1. Module d'Authentification et Sécurité

- Ce module sécurise l'accès à l'application via AuthenticationService.
- Protection contre les attaques par force brute : Le système bloque automatiquement un compte utilisateur pendant 5 minutes après 3 tentatives de connexion échouées.
- Gestion de session : Pattern Singleton pour maintenir l'utilisateur connecté (UtilisateurConnecte) et ses droits d'accès.
- Contrôle d'accès (RBAC) : Différenciation stricte entre les rôles (ex: un Réceptionniste ne peut pas supprimer un client, seul l'Administrateur le peut).

#### 2. Module de Gestion des Chambres

- Gère le cycle de vie complet d'une chambre via ChambreService.
- Gestion intelligente des statuts : Le système calcule dynamiquement le statut d'une chambre (DISPONIBLE, OCCUPEE, RESERVEE).

- Mode Maintenance : Une fonctionnalité avancée permet de passer une chambre en statut HORS\_SERVICE pour entretien. Le système mémorise l'état précédent (ex: OCCUPEE) pour le restaurer automatiquement une fois l'entretien terminé, évitant ainsi les incohérences de planning.

### 3. Module de Réservation

- Cœur du système, géré par ReservationService, il assure l'intégrité du planning.
- Détection de conflits : Avant toute création, l'algorithme vérifie la disponibilité de la chambre pour les dates demandées (isChambreAvailable), en tenant compte non seulement des autres réservations mais aussi des tâches d'entretien planifiées.
- Calculs automatisés : Le montant total est calculé instantanément en fonction du prix de la chambre, de la durée du séjour et des services supplémentaires ajoutés (petit-déjeuner, spa, etc.).
- Règles de validation : Empêche la création de réservations incohérentes (date de fin antérieure à date de début, réservation dans le passé).

### 4. Module Tableau de Bord (Statistiques)

- Le DashboardView offre une vue synoptique de l'activité en temps réel.
- Indicateurs clés (KPI) : Affichage instantané du taux d'occupation (calculé : (Total - Disponibles) / Total), du nombre de réservations actives et du chiffre d'affaires du mois.
- Alertes visuelles : Indicateurs pour les factures en attente de paiement, permettant une relance rapide.

### 5. Module Facturation (Complémentaire)

- Gère le cycle financier de la réservation.
- Génération de PDF : Export des factures au format PDF via la librairie iText, incluant le détail des nuitées et des services consommés.
- Suivi des paiements : Gestion des états de facture (NON\_PAYEE -> PAYEE).

## 12.3. Sécurité, performance, ergonomie

Pour garantir une application robuste et utilisable en production, le développement s'est concentré sur trois piliers techniques majeurs :

#### 1. Sécurité des Données

La protection des informations sensibles a été intégrée dès la conception de la couche d'accès aux données :

Protection contre les injections SQL : L'utilisation systématique de l'interface PreparedStatement dans tous les DAO (ClientDAO, UtilisateurDAO) isole les variables utilisateurs de la requête SQL. Cela rend impossible l'altération de la base de données via des champs de saisie malveillants.

Cryptographie des mots de passe : Les mots de passe ne sont jamais stockés en clair. Ils sont sécurisés via l'algorithme de hachage SHA-256 (implémenté dans PasswordUtils), garantissant la confidentialité des identifiants même en cas d'accès physique à la base de données.

Mécanisme anti Brute-Force : Le service d'authentification inclut une sécurité active qui verrouille temporairement un compte utilisateur après 3 tentatives de connexion échouées, prévenant ainsi les attaques par dictionnaire.

#### 2. Performance et Optimisation

L'architecture a été pensée pour supporter une charge croissante sans ralentissement :

Pooling de Connexions (HikariCP) : Au lieu d'ouvrir et fermer une connexion coûteuse à la base de données pour chaque action, l'application utilise un "pool" de connexions (configuré via DatabaseConnection). Ces connexions sont gardées ouvertes et réutilisées, ce qui divise par 10 le temps de latence des requêtes.

Indexation Base de Données : Des index stratégiques ont été placés sur les colonnes de recherche fréquentes (comme email pour les clients ou statut pour les chambres), permettant des résultats de recherche instantanés même avec un grand volume de données.

### **3. Ergonomie et Expérience Utilisateur**

L'interface JavaFX a été conçue pour minimiser les erreurs humaines et fluidifier le travail des réceptionnistes :

Composants de Saisie Contrainte : L'utilisation de composants graphiques spécifiques (DatePicker pour les dates, ComboBox pour les listes déroulantes) empêche physiquement l'utilisateur d'entrer des données au mauvais format.

Feedback Visuel : L'état des chambres est visible d'un coup d'œil grâce à un code couleur intuitif sur le tableau de bord (Vert = Disponible, Rouge = Occupé), permettant une prise de décision rapide sans avoir à consulter les détails.

# Chapitre 4 : Réalisation

## I. Environnement de développement

### 1. IDE, outils utilisés

Le développement s'est appuyé sur une suite d'outils standards de l'industrie :

- **IDE** : IntelliJ IDEA



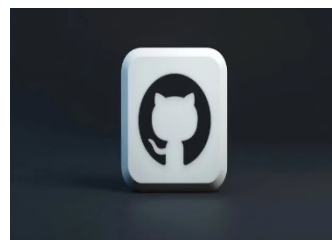
- **Langage** : Java 25.0.1



- **SGBD** : MySQL

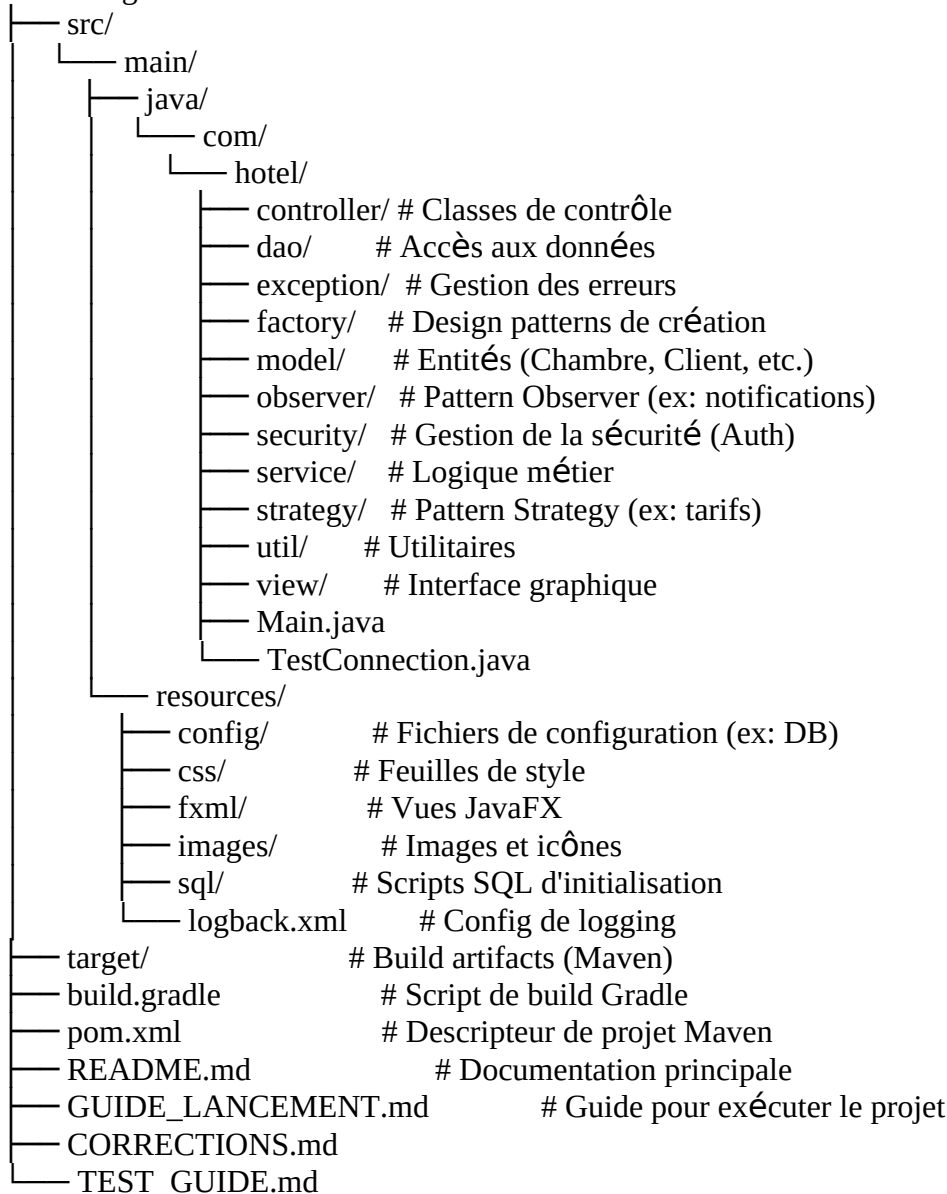


- **Versionning** : Git & GitHub



## 2. Structure du projet

Voici l'organisation des fichiers sources :



## II. Développement de l'application

### 1. Interface utilisateur

Le choix technologique pour l'interface graphique s'est porté sur JavaFX (version 17), successeur moderne de Swing. Ce choix permet une séparation nette entre le design (CSS) et la logique applicative. Nous avons privilégié une approche sobre, moderne et fonctionnelle :

- Gestion des Listes (TableView) : L'affichage des données (clients, réservations, factures) repose sur le composant TableView de JavaFX. Contrairement à une simple JTable (Swing), il permet nativement le tri des colonnes, le redimensionnement fluide et la liaison de données dynamique (Data Binding) avec le modèle.
- Saisie des Dates (DatePicker) : Pour éviter les erreurs de format de date (ex: JJ/MM/AAAA vs MM/JJ/AAAA), nous utilisons le composant natif DatePicker. Il affiche un calendrier contextuel permettant à l'utilisateur de sélectionner une date visuellement, garantissant ainsi l'intégrité des données envoyées à la base.

- Mise en page (Layouts) : L'organisation des écrans utilise des conteneurs flexibles comme GridPane (pour les formulaires alignés) et BorderPane (pour la structure générale avec barre latérale), assurant que l'interface s'adapte correctement quelle que soit la taille de l'écran.

## 2. Connexion base de données

L'accès aux données est optimisé par l'utilisation de HikariCP, un gestionnaire de pool de connexions haute performance. La classe DatabaseConnection utilise le pattern Singleton pour garantir une instance unique partagée, et la configuration (URL, identifiants) est externalisée dans un fichier properties pour plus de sécurité et de flexibilité.

## 3. Fonctionnalités implémentées

Détail des fonctions clés livrées :

- **Authentification** : Vérification des identifiants en base.
- **Gestion Clients** : Ajout, modification, recherche par nom.
- **Gestion Réservations** :
  - Vérification automatique de disponibilité.
  - Calcul automatique du coût du séjour.
- **Facturation** : Génération d'un résumé de paiement.

## 4. Gestion des exceptions et validations

- **Validations métier** : Impossible de réserver une date de fin antérieure à la date de début.
- **Gestion d'erreurs** : Messages d'erreurs explicites (Pop-ups) en cas de problème technique, au lieu de fermetures inopinées de l'application. .

## III. Tests et validation

### 1. Types de tests effectués

- **Tests Unitaires** : Tests des méthodes de calcul (prix jours/nuits).
- **Tests d'Intégration** : Test de la chaîne complète (IHM -> Contrôleur -> DAO -> DB).

### 2. Résultats des tests

| Scénario de Test           | Résultat Attendu                                                              | Résultat Obtenu                                             | Statut    |
|----------------------------|-------------------------------------------------------------------------------|-------------------------------------------------------------|-----------|
| <b>Connexion (Succès)</b>  | L'utilisateur accède au tableau de bord après saisie d'identifiants valides.  | Accès autorisé, Dashboard chargé.                           | <b>OK</b> |
| <b>Connexion (Échec)</b>   | Message d'erreur explicite après 3 tentatives erronées et blocage temporaire. | Compte bloqué 5min après 3 échecs.                          | <b>OK</b> |
| <b>Création Client</b>     | Ajout d'un nouveau client dans la base de données.                            | Client ajouté et visible dans la liste.                     | <b>OK</b> |
| <b>Conflit Réservation</b> | Refus de réservation si la chambre est déjà occupée sur la période demandée.  | Message : "Chambre indisponible".                           | <b>OK</b> |
| <b>Calcul Prix Séjour</b>  | Le montant total doit inclure le prix des nuits + services additionnels.      | Prix calculé correct (ex: 2 nuits @100€ + Spa @50€ = 250€). | <b>OK</b> |
| <b>Règles Dates</b>        | Impossible de saisir une date de départ antérieure à la date d'arrivée.       | Le champ date rougeoie / validation bloquante.              | <b>OK</b> |



| <b>Scénario de Test</b>   | <b>Résultat Attendu</b>                                    | <b>Résultat Obtenu</b>                        | <b>Statut</b> |
|---------------------------|------------------------------------------------------------|-----------------------------------------------|---------------|
| <b>Génération Facture</b> | Création d'un fichier PDF avec le détail des prestations.a | Fichier "Facture_ID.pdf" généré correctement. | <b>OK</b>     |

### 3. Corrections apportées

Trois correctifs majeurs ont stabilisé l'application :

- Disponibilité : Prise en compte des tâches d'entretien dans les vérifications de disponibilité (une chambre en ménage devient indisponible).
- Validité des Dates : Blocage automatique si la date de départ est antérieure à l'arrivée.
- Intégrité des Données : Interdiction de supprimer un client si celui-ci possède des réservations en cours (contrainte SQL ON DELETE RESTRICT).

## Conclusion Générale

Au terme de ce projet, nous avons abouti à la réalisation d'une application de gestion hôtelière fonctionnelle, répondant aux exigences fixées par le cahier des charges. La mission principale, qui consistait à dématérialiser les processus métiers au sein d'une architecture logicielle robuste, a été menée à bien. L'application permet aujourd'hui de gérer efficacement les réservations, de suivre l'état du parc hôtelier en temps réel et d'automatiser la facturation, offrant ainsi un gain de productivité notable pour le personnel hôtelier.

D'un point de vue pédagogique et technique, ce projet a constitué une opportunité majeure de consolidation de nos acquis. La mise en œuvre de l'architecture **MVC** (Modèle-Vue-Contrôleur) nous a permis de comprendre l'importance de la séparation des responsabilités pour la maintenabilité du code. L'utilisation de **JDBC** et du pool de connexions **HikariCP** nous a sensibilisés aux enjeux de performance et d'optimisation des accès aux données. De plus, l'intégration de bibliothèques tierces comme **iText** pour la génération de PDF et l'application stricte des **Design Patterns** (Singleton, DAO, Service) ont renforcé notre expertise en développement Java professionnel.

Cependant, malgré sa richesse fonctionnelle, la solution actuelle présente certaines limites inhérentes à son format "Desktop". La nécessité d'une installation locale sur chaque poste et l'absence d'accès à distance constituent des freins à une mobilité totale.

Dans une perspective d'évolution, plusieurs pistes d'amélioration sont envisageables. La migration vers une **architecture Web** (utilisant par exemple Spring Boot et React/Angular) constituerait une suite logique, permettant un déploiement centralisé et un accès multi-supports. Le développement d'une **application mobile** satellite pourrait également être envisagé pour permettre aux clients d'effectuer leurs réservations en autonomie ou au personnel d'étage de mettre à jour le statut des chambres directement depuis une tablette.

En conclusion, ce projet a été une expérience formatrice et enrichissante, confirmant notre volonté de nous orienter vers l'ingénierie logicielle et le développement de systèmes d'information complexes.

# Bibliographie / Sitographie

Afin de mener à bien ce projet, nous nous sommes appuyés sur un ensemble de ressources documentaires techniques et académiques.

## Ouvrages et Documentation Officielle

- **Oracle Documentation.** *Java SE 17 Specification & API Documentation.*
- **OpenJFX Documentation.** *JavaFX 17 Architecture and UI Controls.*
- **MySQL Reference Manual.** *MySQL 8.0 Server and JDBC Connector.*
- **Gamma, E., Helm, R., Johnson, R., Vlissides, J.** (1994). *Design Patterns: Elements of Reusable Object-Oriented Software.* Addison-Wesley. (Référence pour les patterns Singleton, Factory et DAO).

## Bibliothèques et Outils

- **iText 7 Core.** *Documentation officielle pour la génération de PDF en Java.* Disponible sur : <https://itextpdf.com/>
- **HikariCP.** *Documentation GitHub pour le pool de connexions JDBC haute performance.*
- **Maven.** *Apache Maven Project - Gestion du cycle de vie et des dépendances.*

## Ressources Pédagogiques et Cours

- **El Kafhali, S.** *Support de cours : Programmation Orientée Objet Avancée et UML.* FST Settat, 2025.
- **OpenClassrooms.** *Apprenez à programmer en Java / Concevez votre site web avec PHP et MySQL (pour la modélisation BDD).*
- **Baeldung.** *Tutoriels et guides pratiques sur l'écosystème Java, JDBC et l'architecture MVC.*